Mechanical Engineering Senior Design Project
(ME SDP)

ME SDP 117: Brain-controlled Drone

Design Report

Sponsor Dr. Zhou, Jian

Date 05/06/2026

Submitted by:
Christian Tai
Bhavi Patel
Rowan Hawk
Partho Nath
Lily Silva

Faculty Advisors:
Professor, Selleck, Colin
Dr. Zhou, Jian
Dr. Zhao, Jingzhou

Table of Contents

Executive Summary 3
1. Introduction 3
2. Problem Definition 4
2.1 Problem Scope 4
2.2 Technical Review 4
2.3 Design Requirements 5
3. Design Description 6
3.1 Overview 6
3.2 Detailed Design Description 7
3.2.1 Camera Tripod 7
3.2.2 Motion Tracking System 7
3.2.3 Cyton BCI Board 9
3.2.4 Brainflow and Interfacing with Cyton 10
3.2.5 DJI Tello Drone 13
3.3 Brain Controlled Drone System 14
4. Implementations 19
4.1 Brain Controlled System 19
4.2 Motion Tracking System 20
5. Testing and Evaluation 21
6. Schedule and Finances 24
6.1 Finances 24
6.2 Gantt Chart 25
7. Social Impact 25
8. Project Operation 26
8.1 How To Operate Drone Control System 26
8.2 How to Operate Motion Tracking System 32
9. Bibliography 33
Appendix A 34
Appendix B 34

Executive Summary

The ME SDP 117 is a drone that you control with your mind. There are electrodes on your head and a Python software that interprets your brain's activity instead of a remote control or joystick. The project consists of two subsystems that operate in tandem: one that tracks the drone's location in physical space and another that detects user biosignals and translates them into flight orders.

The OpenBCI Cyton Board, an 8-channel board that reads EEG, ECG, and EMG data at microvolt resolution, powers the signal side. In accordance with the 10-20 international system, electrodes are positioned at particular scalp locations, each of which targets a distinct signal. The actual job is done by three signals: EOG spikes detect intentional blinks, beta waves represent concentration level, and alpha waves show whether the eyes are open or closed. The user receives four commands from those three inputs: takeoff, land, move forward, and rotate. The drone that receives those commands is a DJI Tello, which was chosen due to its Python programming capability, integrated propeller guards, and Wi-Fi connectivity to a laptop.

The tracking side uses two e-CAM20_CUOAGX cameras connected to an NVIDIA Jetson AGX Orin. The cameras capture synchronized stereo frames, and the Jetson runs a fine-tuned YOLOv8n model to detect the drone in both views. After stereo calibration and rectification, disparity between the left and right detections is used to estimate the drone’s approximate relative position in real time. Additional labeled images from cluttered lab scenes and the final demo location were used to fine-tune the detector so that it would operate more reliably in realistic indoor environments.

On budget: the project ran over the original $1,700 cap, which was never realistic given the hardware. The Cyton Board is $1,249 on its own. The DJI Tello adds another $275. The final total came to $2,136.40, against a revised allocation of $4,000, with cameras, the Jetson, and some hardware provided by Binghamton University directly.

Most of the semester's testing time went into signal consistency finding electrode positions that gave clean reads, tuning noise filters, and setting thresholds that worked across different users without constant false positives. That problem is solvable. The final prototype demonstrates it.

1. Introduction

The purpose of this project is to design, build, and program a drone that will be controlled by one's mind. The SDP 117 team was tasked with making a mind control drone that will use EEG electrodes to collect signals to give instructions to a drone. This project will be using pre-existing materials such as a drone from DJI and a Cyton Board from OPEN BCI. The Cyton Board is a board that has 8 channels to be used to collect EEG, ECG, EMG, and EOG signals. This board is one of the main components used in this project as it is used to collect EEG signals from a user's head.

EEG (electroencephalography) picks up the tiny voltage changes produced by electrical activity in the brain. Electrodes sit at specific spots on the scalp (placed according to the 10-20 international system) and the Cyton amplifies and digitizes what they detect. The raw signal gets broken into frequency bands. Alpha waves (8–13 Hz) are what you see when someone closes their eyes and relaxes; they drop off as soon as the eyes open. Beta waves (13–30 Hz) go up during focused concentration. The system uses alpha power to detect eye state and beta power as a proxy for intent or engagement.

EOG (electrooculography) works differently. The eye has a natural electrical potential between the cornea and the retina, so when the eye moves or the eyelid closes, nearby electrodes pick up a voltage shift. Electrodes above and below the eye catch blinks; electrodes to the side can catch horizontal gaze. In this system, blinks from EOG serve as discrete commands, a way for the user to send a signal without moving anything except their eye.

Those three streams alpha state, beta level, EOG blinks get combined to drive the drone. Getting that to work took real testing: trying different electrode positions to find where each user's EEG was cleanest, filtering out noise and movement artifacts, and figuring out what thresholds actually produced consistent results versus false positives.

The project also has a motion tracking system running alongside all of this, using Nvidia Jetson cameras to track where the drone is. That subsystem required substantial setup and integration effort, including Jetson configuration, camera driver installation, stereo calibration, and the design of a custom camera mounting solution. The designs are included in this report.

2. Problem Definition

2.1 Problem Scope

The problem is to develop a non-invasive, user-friendly, brain-machine system that enables users to control a drone's movements using biopotential signals. As said in the project requirements, the drone must acquire, process, and interpret EEG, or EMG signals in real time to generate at least three separate drone commands. In addition to a bio signal-based control system, the drone must also incorporate motion tracking using cameras to display the drone's 3D position in real time during flight. By utilizing AI hardware and cameras, the design will be able to track the drone as it changes locations in response to user commands.

2.2 Technical Review

Brain controlled interfaces (BCI's) are a well-researched area of exploration that involve obtaining brain signals with electrodes that capture the electrical activity from neurons in the brain. The ability to record these biopotential signals through electroencephalography (EEG) can be done both invasively, by implanting electrodes, or non-invasively by using superficial electrodes attached to the scalp. Early development of brain-controlled interfaces started with studies on animal models, that allowed research to explore neural decoding systematically.

In the 1960's and 1970's, scientists were able to prove that electrical activity recorded from monkeys, could be mapped to specific motor intentions. The firing rate of individual neurons in monkeys was able to control a simple device. Another researcher, Eberhard Fetz, showed that monkeys were able to control meter needles using neural activity. And as recording technology improves, scientists were able to map the activity of groups of motor cortex neurons

to robotic arm movements. These early animal studies were essential for laying the groundwork for decoding algorithms and signal-processing for modern-day BCI's.

As interest grew in bridging BCI's with human users, research invested more time into non-invasive recording methods that are used on this project. EEG became the dominant human-centered approach for these models. Current EEG-based BCI systems can detect reliable patterns such as motor imagers, steady-state visual evoked potentials, and spikes in neural activity from a target stimulus. These signals are processed using established methods, including Fourier-based analysis, common spatial patterns, and band-pass filtering. These neural patterns became the basis for navigating simple robotic systems.

Several commercial devices demonstrate how BCIs are implemented today. One notable company is Emotiv, which produces a lightweight, wireless, EEG headsets, that include onboard filtering. However, these devices use closed-source classification algorithms which prevent users from altering filter settings, adjusting the feature extraction methods, or implementing custom real-time decoding strategies. While the headset is convenient, and user-friendly, it lacks customization and experimental flexibility that this project provides.

In addition to bio signal interfaces, there is also existing work on the technology needed to operate and track a drone. Most programmable drones communicate through standardized methods such as Wi-Fi or Bluetooth, allowing a computer to send basic flight commands.

To monitor where a drone is in space, many systems rely on cameras or computer-vision tools that can detect the drone's location in real time. These tracking methods are commonly used in robotics labs, but they are rarely combined with brain-controlled systems. As a result, most existing BCI-drone demonstrations do not include an integrated way to track the drone's motion or provide real-time feedback, which limits precision and control.

Additionally, existing BCI-controlled drone experiments also tend to rely heavily on EMG signals rather than cognitive EEG features and rarely do they incorporate real-time motion tracking and closed loop feedback. This level of control is essential for drone operations where command reliability and responsiveness are at the forefront of the functionality of the project.

Altogether, the prior shows that while BCIs and motion tracking are well established individually, integrating them into a unified system remains to be an unresolved challenge.

2.3 Design Requirements

The first requirement that is given for this project is that the drone will be able to at least take-off and hover. When the brain is at a rest and calm state, this will communicate with the drone to stay on the ground and to not move. When the brain is at a focused state, this will communicate to activate the drone and to activate flight mode. Once the drone is off the ground, the next requirement of this project is to have the drone fly in a confined space. Team SDP 117 does not want the drone to be flying out of control which is why there will be a safety switch included to keep the drone in a confined space to prevent the drone from crashing into a nearby wall.

This leads to one of the next requirements which is to not have the drone crash into anything. It would be bad to have a drone that goes out of control and crashes. The drone propellers have blade guards that protect the blades and anyone or anything that the drone might inadvertently bump into.

Another requirement that is included in this project is that the Cyton board will also be able to acquire, process, and display biopotential signals to extract to be able to control the

drone. Commands can then be sent to the drone based on the EEG signals.

There are also a couple of stretch goals for this project. The drone should be able to utilize multiple channels of biopotential signals. Each signal should act as a switch, which leads to the next stretch goal of having the brain control drone be able to travel in all directions. The last stretch goal is that the drone should be completely controllable with EEG signals and not rely on any EMG muscle control.

3. Design Description

3.1 Overview

The purpose of the design is to create a brain-controlled drone interface by sending EEG signals from the brain to be processed into code while its position is being tracked in real time. To complete this task, the design is broken down into separate parts: the brain signals, the tracking, and the drone. Each of these components can work independent from each other but use each other's signal and data to process information.

The main component of our design is the OpenBCI Cyton Board. This 8-channel board has the capabilities of reading EMG, ECG, and EEG signals with astonishing precision. The Cyton Board reads signals by connecting electrodes to the different information reading channels as well as bias and reference channels to measure the voltage differences between the waves our brain emits. Comb electrodes will be used that allow for signals to be received past hair, an obstacle encountered during the planning phase of the design. The Cyton Board receives the signals from the electrodes and sends them to a laptop through a Bluetooth dongle. The data can then be processed and plotted to understand how different actions impact the voltages the brain emits. Through previous research on the signals that the brain emits, the key action that can be detected easily is concentration, relaxation, and closing the eyes. Concentration increases the beta waves and decreases the alpha waves that the brain emits whilst relaxation does the opposite. The differences between these actions will be used as a switch to later control the drone. Closing eyes increases the power of alpha waves because blocking visual input allows the brain to allocate more resources into internal actions and will also be used as a control for the drone. Different electrode placements allow for more accurate readings. By connecting the electrodes to different channels and placing the electrodes on the different key parts of the head, the Cyton Board can effectively capture multiple signals that can be used to pilot the drone in real time. The elements of the brain signal collection were all purchased from the same manufacturer to ensure compatibility.

After sending the processed signal to the computer, code is used to process the signal into a more useful form. By calculating averages, transforms, and cycles of the signal, the signals can be used to establish conditions for the drone to complete certain tasks. The drone of choice is the DJI Tello, a programmable small drone that can use python commands as a controller. By setting the condition for takeoff to be a peak in the alpha waves, for example, the user can relax, hence increasing alpha waves so the drone will take off. This method will be used for further movement of the drone, where the more precise and reliable signals to be used for the more intricate parts such as turning and flipping.

To track the drone’s location in real time, the design uses a stereo-vision subsystem built around two e-CAM20_CUOAGX cameras and an NVIDIA Jetson AGX Orin. The cameras capture synchronized left and right images, and after stereo calibration these images are rectified

so the system can compare corresponding features between both views. A fine-tuned YOLOv8n object-detection model running on the Jetson identifies the drone in each frame, and stereo disparity information is then used to estimate the drone's approximate relative X, Y, and Z position. Because camera alignment is critical for this process, a custom tripod and aluminum-extrusion mount was designed so the stereo pair could be adjusted, fixed in place, and recalibrated as needed. After initial testing, additional labeled images from cluttered indoor scenes and the final demo location were used to fine-tune the detector, which significantly improved tracking reliability in realistic environments.

3.2 Detailed Design Description

3.2.1 Camera Tripod

The specific cameras that work with the NVIDIA Jetson are incredibly powerful and compact with a unique design which standard designs do not work with. Therefore, to keep these powerful cameras safe and adjustable, a camera tripod needed to be created. Through multiple iterations, the camera design needed to be secured onto a steady base yet adjustable, so an aluminum extrusion was the ideal element for the base. In previous designs, a non-stable, bulky material led to adverse results. By designing the bottom portion of the tripod to fit onto this aluminum frame, it is capable of sliding steadily in the hole of the aluminum extrusion yet also be tightened to lock the tripod into place. The bottom portion also has guiding extrusions to prevent rocking and rotation. The camera is small, so the tripod's tolerance was an important element. The holder was designed to fit the camera securely while minimizing looseness in the mount, which reduced unwanted movement during testing and calibration. The camera tripod is 3D printed in PLA because of its flexibility as well as cost. The low material cost allowed multiple design iterations so the holder could be adjusted to fit the camera properly while still leaving more of the project budget available for major system components such as the Cyton Board and the DJI Tello Drone. Although the holder itself is plastic, steel screws are still used to secure the tripod into place, which helps keep the full assembly rigid. Rubber bands were used to secure the cameras in place within the 3D printed holders, as the alignment of the cameras staying exact are critical to the system.

In the final motion-tracking system, this tripod assembly served as more than just a camera holder. The stereo tracking pipeline depended on both cameras maintaining a fixed baseline and stable orientation after calibration, so the rigidity of the tripod and aluminum extrusion directly affected repeatability of the motion-tracking results. The sliding feature of the extrusion also allowed adjustment of the stereo setup for different testing spaces and then lock the cameras in place once the desired field of view was reached. As a result, the tripod became a necessary mechanical part of the full motion-tracking subsystem rather than only a mounting accessory.

3.2.2 Motion Tracking System

The motion-tracking subsystem was designed to perform real-time visual tracking of the drone using a pair of e-CAM20_CUOAGX cameras connected to the NVIDIA Jetson AGX Orin. These cameras interface with the Orin through two independent 2-lane MIPI CSI ports, allowing synchronized image capture for stereo-vision processing. Unlike a single-camera setup, this system uses two simultaneous camera streams so that the positional difference of the drone across the left and right images can be used to estimate depth and relative position in three-

dimensional space. To support this configuration, the Jetson AGX Orin was flashed with JetPack 5.0.2 and operated on L4T 35.1.0 so that the camera drivers, multimedia libraries, and supporting software remained compatible with the hardware provided by the manufacturer.

In addition to the camera interfaces, the NVIDIA Jetson AGX Orin required several external connections for system operation, development, and debugging. Power was supplied through the dedicated DC power connector using the manufacturer-specified adapter to maintain stable operation under high computational load. A monitor was connected through DisplayPort to provide access to the Jetson desktop environment, while a USB keyboard and mouse were used for local setup and testing. Ethernet was also used during development for software installation, file transfer, and remote communication. The cameras themselves were mounted using the custom tripod and aluminum-extrusion system described in Section 3.2.1 so that both sensors could be held securely and aligned consistently for stereo calibration. Maintaining a rigid and adjustable camera mount was important because small changes in camera position directly affected stereo rectification and depth estimation. The cameras were mounted at a fixed baseline on the tripod assembly once calibration was complete.



Figure 1: Jetson Hardware

Once the Jetson and cameras were prepared, the manufacturer's camera drivers were installed by applying the required device-tree support, kernel modules, and related software dependencies. Diagnostic tools such as *dmesg* and *v4l2-ctl* were used to verify that both cameras initialized properly and streamed through the MIPI CSI pipeline. After hardware bring-up, the stereo pair was calibrated so that the intrinsic camera parameters, relative orientation, distortion terms, and baseline between the cameras could be measured. These calibration results were then used to rectify the two camera streams so that corresponding image features aligned properly between the left and right frames.

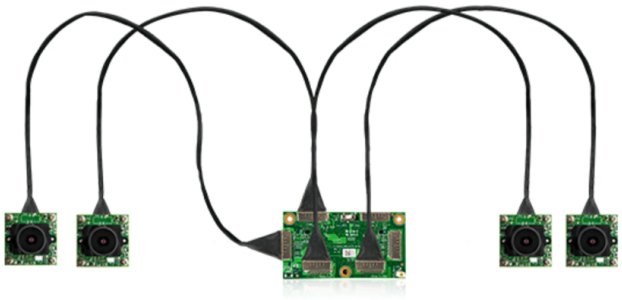


Figure 2: Jetson Cameras

The final motion-tracking program was implemented in Python using OpenCV-based stereo processing together with a fine-tuned YOLOv8n object-detection model running on the Jetson. In each frame, the detector identified the drone and produced a bounding box in the image. The system then used the drone location in the stereo pair together with disparity information to estimate the drone's approximate X, Y, and Z position relative to the stereo camera frame. Rather than relying on raw pixel comparisons alone, the final pipeline combined stereo rectification, object detection in both views, and disparity-based depth estimation so that the system could continue tracking the drone while it moved through the scene. The interface displayed the detection output alongside a disparity map so that tracking performance could be monitored visually during operation.

During development, early versions of the detector performed well against simple backgrounds but struggled more in cluttered indoor scenes and at greater distances. To improve performance, the team expanded the training dataset by collecting and labeling additional images of the drone in realistic lab environments, including cluttered backgrounds and the final demo location. These images were then used to fine-tune the YOLO model so that the detector better matched the actual operating conditions of the project. This significantly improved the system's ability to continue detecting the drone without requiring a plain white background behind it. The completed motion-tracking subsystem was therefore able to provide reliable real-time drone detection and tracking during demonstration. While the stereo pipeline generated position values that changed consistently with drone movement, the coordinate output was not treated as a precision measurement system. Instead, the final subsystem successfully met the project need of visually locating and tracking the drone in real time while supporting the broader brain-controlled drone demonstration.

3.2.3 Cyton BCI Board

For this project the Cyton BCI from Open BCI (Brain Computer Interface) will be used for interfacing with brain wave signals. This board allows the reading and processing of EMG, ECG, and most importantly EEG data. The board uses a small battery and communicates to a USB dongle with Bluetooth via a RFD2230, a low energy Bluetooth sender and receiver. This allows the board to wirelessly communicate with a computer, meaning that the pilot can have the board affixed to themselves and move freely. The Cyton BCI is an eight-channel circuit board and features accuracy in the microvolt range, extremely important for reading the relatively small

voltage differences that can be measured at the surface of the pilot's head. It also outputs data at a rate of 250 Hz, plenty for applications needed in this project, as the brainwaves we will look at will be within the 0-100Hz range.

This is possible due to the special analog to digital converter featured on the board, the ADS1299. This ADC is specially designed for biopotential sensing applications in mind. In addition to the main eight channels, each leading to a positive and negative input pin, there is an SRB2, and the BIAS pin. The SRB2 acts as a common reference for all eight channels; it shorts to the positive pin using the ChxSET[3] command set to True. This is especially needed for the EEG setup, as all electrodes will be referenced to a common point that is placed somewhere with minimal interference from biopotential signals such as the earlobe.

The BIAS pin acts as noise reduction; this is common in biopotential applications because the human body can act as an antenna that picks up interference from electronics. This will often be in the form of a 60Hz sine wave that can be in the range in the hundreds of micro volts. The BIAS runs to an internal *right leg drive.* This circuit reduces the noise by dynamically driving back voltage into the body depending on the input noise.

3.2.4 Brainflow and Interfacing with Cyton

Open BCI has a GUI that is useful for a set up view of the data. However, to get the data from the Cyton directly and process it ourselves we are using the command library *Brainflow.* Brainflow allows for easy communication between a programming language such as python and the Cyton board. With a couple simple set up commands, such as inputting the board ID and the port the USB dongle is attached to, data from the Cyton Board can be received directly. That can then be processed for analysis and eventually trigger a drone command using the djitellopy library.

```
34  params = BrainFlowInputParams()
35  params.serial_port = 'COM3'
36  board_id = 0
37  board = BoardShim(board_id,params)
38  #df.FilterTypes(0)
39  board.prepare_session()
40
41  print("board turing on... RELAX")
42  time.sleep(3)
43  print("press space to start stream")
44  while(not kb.is_pressed("space")): pass
45  print("starting stream")
46  board.start_stream()
47  print("calibrating..RELAX")
48  time.sleep(3)
49  calibration_data = board.get_board_data()
50
51  for i in range(8):
52   time.sleep(1)
53   print(i+1)
54
55  data = board.get_board_data()
56  print("ending stream")
57
58  board.stop_stream()
59  board.release_session()
60
61  emg_channels = board.get_ecg_channels(board_id)
62  print(emg_channels)
63  print(data[1])
64  starting_point = np.mean(calibration_data[1])
65  print(calibration_data)
```

Figure 3: Cyton Code

```
for i in range(8):
    time.sleep(1)
    print(i+1)

data = board.get_board_data()
print("ending stream")

board.stop_stream()
board.release_session()

emg_channels = board.get_ecg_channels(board_id)
print(emg_channels)
print(data[1])
starting_point = np.mean(calibration_data[1])
print(calibration_data)
print(starting_point)
channel5 = np.delete(data[1], obj: 0) #this removes 1st da
normal_channel5 = channel5 - starting_point
channel_data = normal_channel5

channel_data.tofile( fid: 'channel_data.csv', sep=',')

graph_with_fft(normal_channel5)
```

Figure 4: Continuation of Cyton Code

To test the capabilities of the cyton board and confirm the data was correctly being pulled from the board with brain flow, initial EMG, ECG, and EEG tests were conducted. The first started with measuring EMG signals as they are the easiest to see. Electrodes have been placed on top of the forearm, and the biopotential signal was measured as the forearm rested and flexed. Below in order is a control (no flex), flex solid (after 2 seconds began flexing the forearm), and flex cycle (short flexes every 2 seconds).

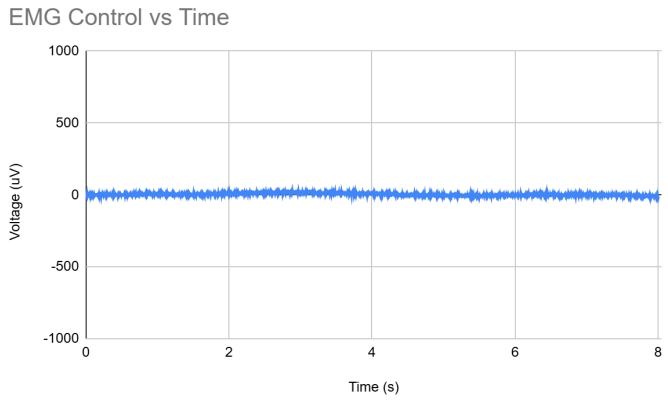


Figure 5: EMG Control VS Time

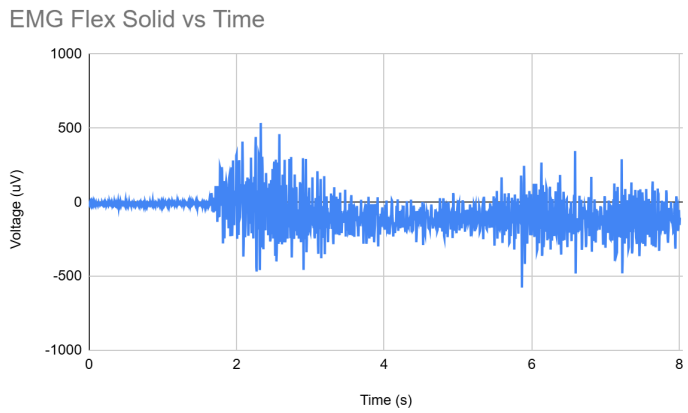


Figure 6: EMG Flex Solid VS Time

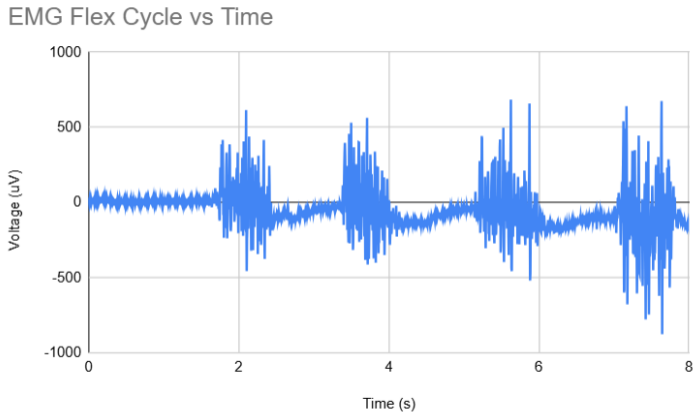


Figure 7: EMG Flex Cycle VS Time

Next ECG signals were tested. A typical 3 lead system for measuring the potential difference generated across a person's chest as the heart beat. To show comparison, a resting heartbeat and a raised one were measured.

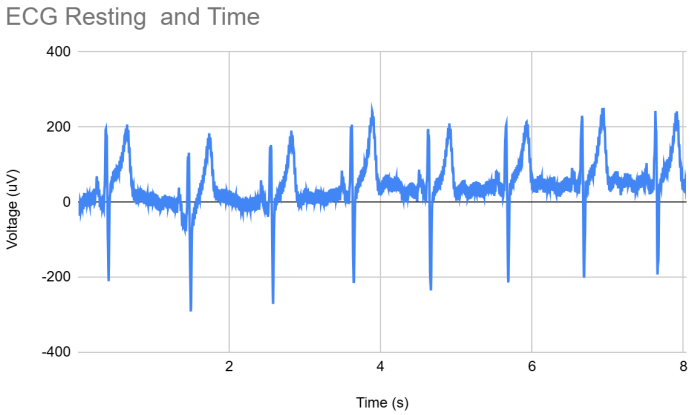


Figure 8: ECG Resting time

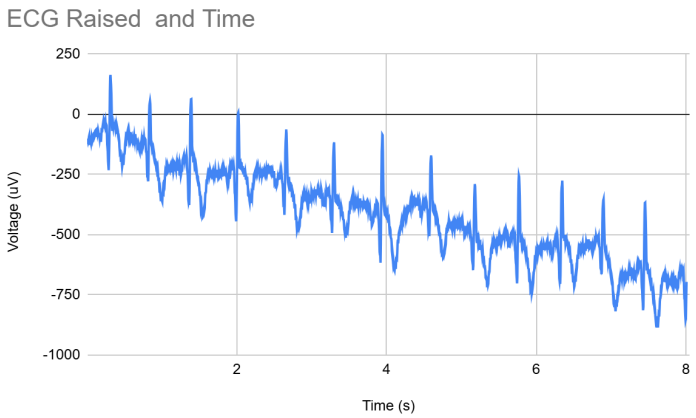


Figure 9: ECG Records After Doing Physical Activity

Lastly, EEG signals were tested. Over a period of 8 seconds, EEG signals from O1 (located on the back of the head) were measured, with eyes open and with eyes closed to compare. This data has been passed through a bandpass filter to only show the alpha wave range (8Hz-12Hz). In theory, eyes closed should show higher amplitude alpha waves, which holds for the first 6 seconds of the trial. More analysis to be done to get consistent results.

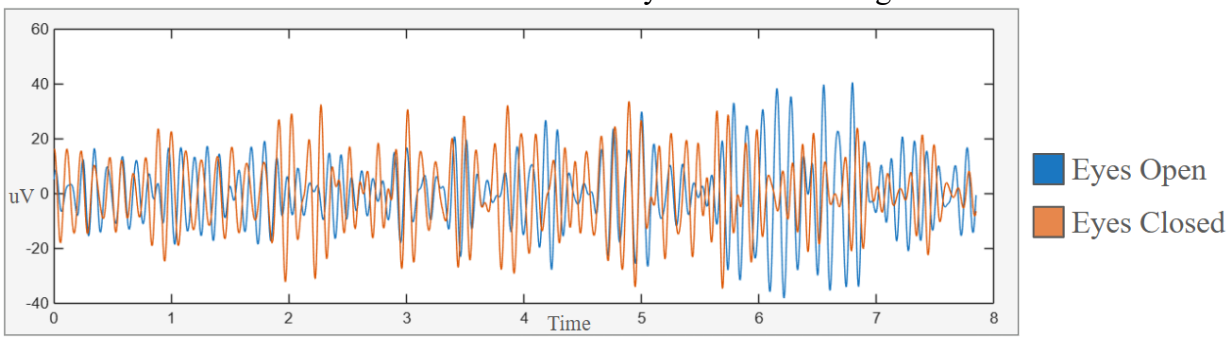
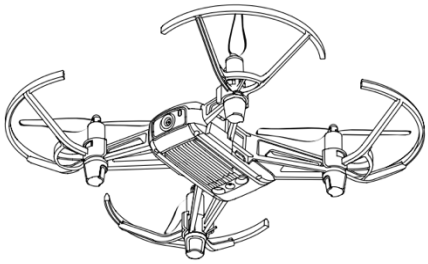


Figure 10: EEG Signals of Eyes Open VS Eyes closed Data in Fall 2025

3.2.5 DJI Tello Drone

The drone that was selected to use in this project is the DJI drone from Tello. This is a quadcopter that has live position tracking allowing an easy way to hover in place using a vision-based positioning system. It can monitor its height between about 0 and 20ft high, plenty for this project. This drone was selected because of its ease of use when it came to programming projects with the djitellopy library. This library allows easy communication and control between a computer and the drone. Simple commands in python can make the drone move a certain number of centimeters in any direction or rotate. The drone has only a 13-minute battery life, however battery packs are cheap and easy to charge and swap out meaning that a cease of operation due to a dead battery will never last more than a moment



```
from djitellopy import Tello

tello = Tello()

tello.connect()
tello.takeoff() # tello drone will takeoff and rise 80cm

tello.move_left(100) # tello drone will move left 100cm
tello.rotate_clockwise(90) # tello drone will rotate clockwise 90 degrees

tello.land() #tell drone will land
```

Figure 11: DJI Tello Drone

3.3 Brain Controlled Drone System

The brain-controlled system uses three bio signal inputs; alpha waves, beta waves, and EOG signals. Each one maps to a specific flight command, and together they give the user full control over the drone using only brain activity and eye movements.

Alpha Waves - Takeoff and Landing

Alpha waves fall in the 8–12 Hz range. They're strongest over the occipital lobe, which is why the O1 electrode in figure 12 is placed there that position picks up the clearest alpha signal. The key behavior the system exploits is that alpha power increases when the eyes close and falls off when they open. The team's own data in figure 13 backs this up: eyes-closed readings averaged around 6.5 μV^2 compared to about 4 μV^2 eyes-open. That's a consistent gap across trials.

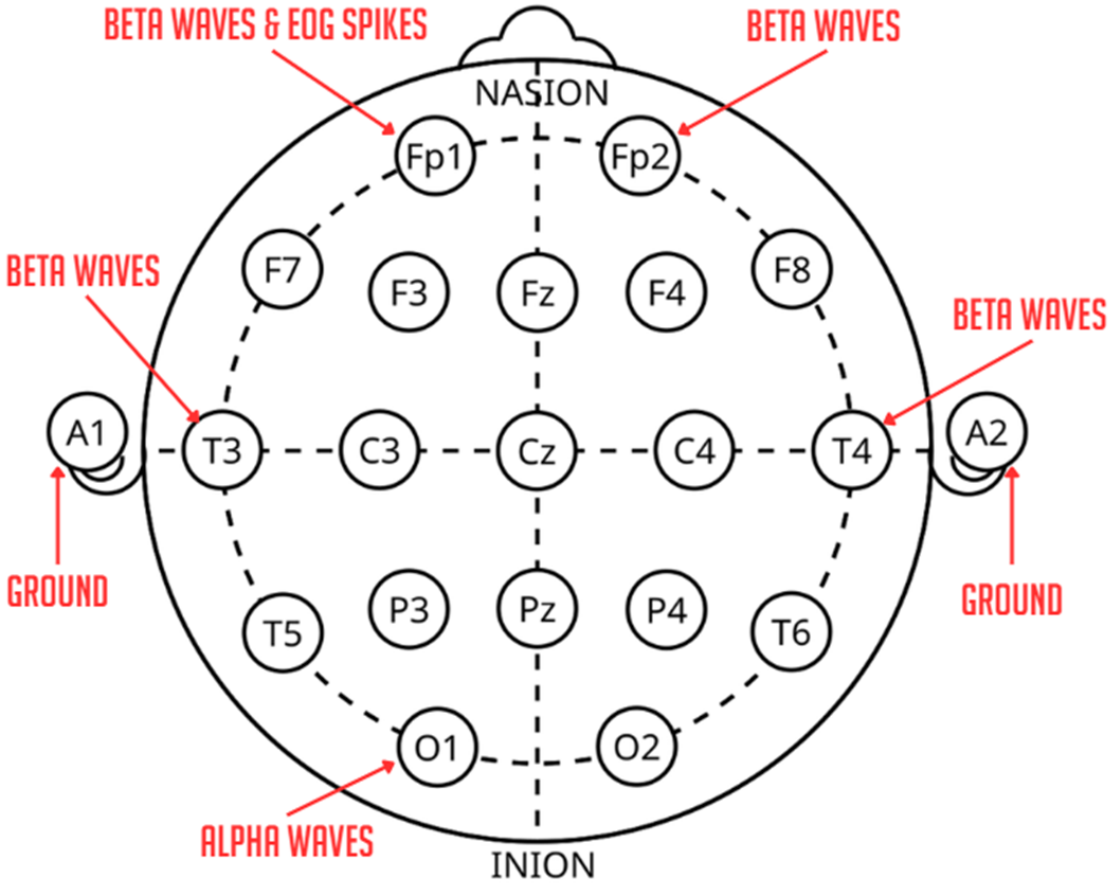


Figure 12: Brain Map

Because alpha amplitude varies between users, the system runs a short calibration at the start of each session to set that person's baseline threshold. Noise can still push occasional false spikes through, so a single elevated reading does nothing on its own. Five consecutive eyes-closed readings in a row are required to trigger a command. To take off, the user closes their eyes for about 10 seconds until those 5 readings register. Landing works the same way the drone is in the air, the user closes their eyes, and 5 consecutive readings bring it down.

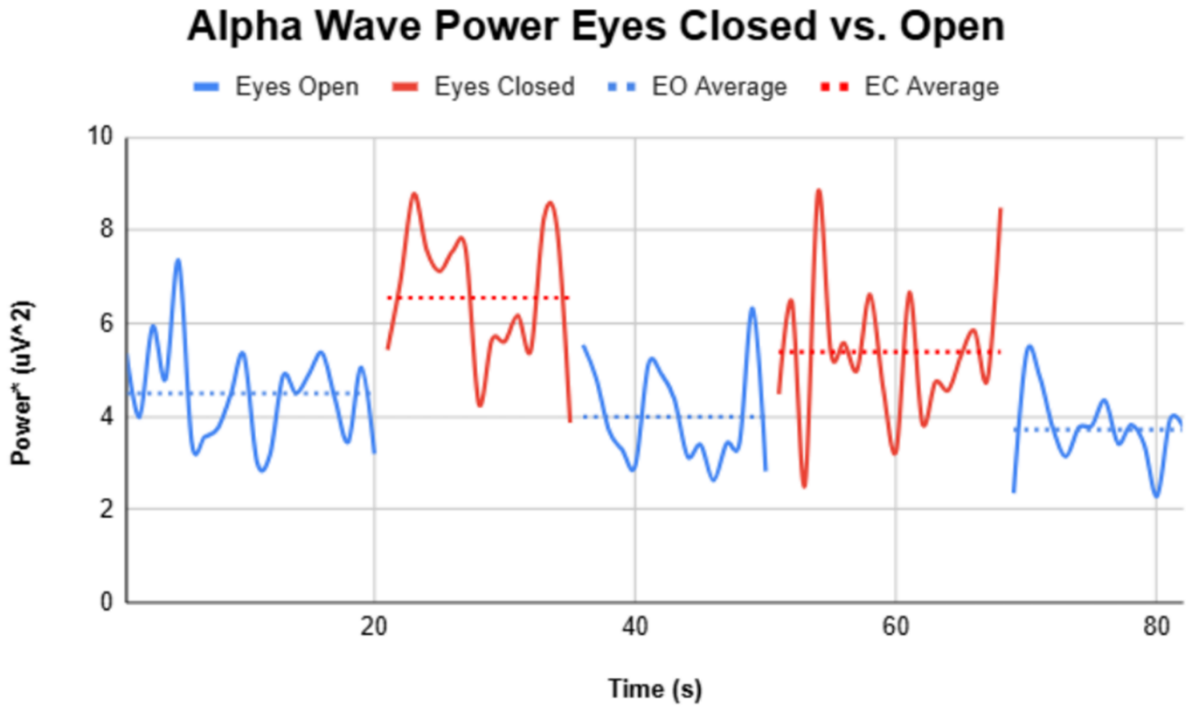


Figure 13: Final Data of Eyes Open VS. Eyes Closed

EOG Signals - Double Blink Commands

EOG signals come from the Fp1 electrode, positioned above the eye on the forehead. A blink shifts the Corne retinal potential and the Cyton picks it up as a sharp voltage spike. Figure 14 shows this clearly: the signal is mostly low-amplitude noise, then a blink produces a peak-trough pair around ±500 µV. The problem is people blink involuntarily all the time, so a single blink can't be used as a command. The system detects double blinks instead, which are uncommon enough in normal behavior to serve as a deliberate input.

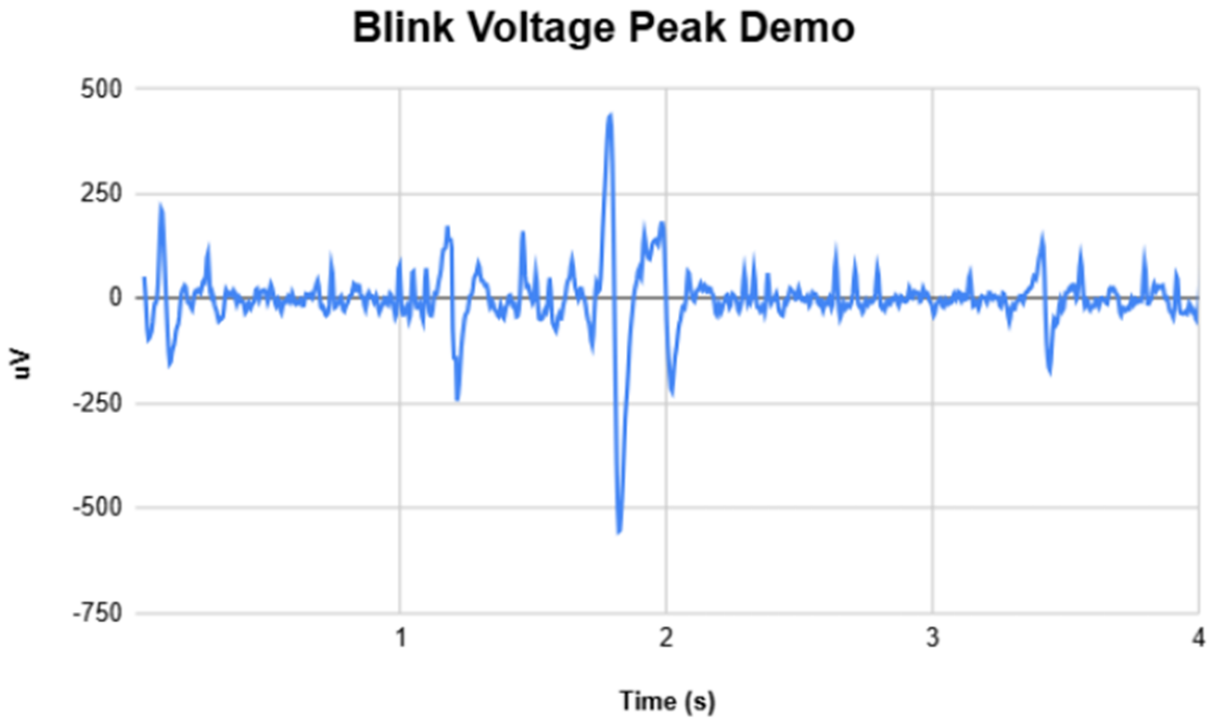


Figure 14

Beta Waves - Concentration State

Beta waves run in the 13–30 Hz range and reflect concentration level. The electrodes at Fp1, Fp2, T3, and T4 cover the frontal and temporal lobes, which are most active during focused mental effort. figure 15 shows seven trials using Brainflow's built-in machine learning model to score concentration. In every trial, the concentrated state scored higher than the relaxed state; relaxed values ranged from 0.53 to 0.72 while concentrated values came in between 0.75 and 0.99. The model separates the two states reliably.

EOG and beta signals combine into two directional commands. A double blink while concentrating moves the drone forward. A double blink while not concentrating rotates it 90 degrees counterclockwise. The double blink requirement means the concentration state alone never accidentally triggers anything both inputs have to be present.

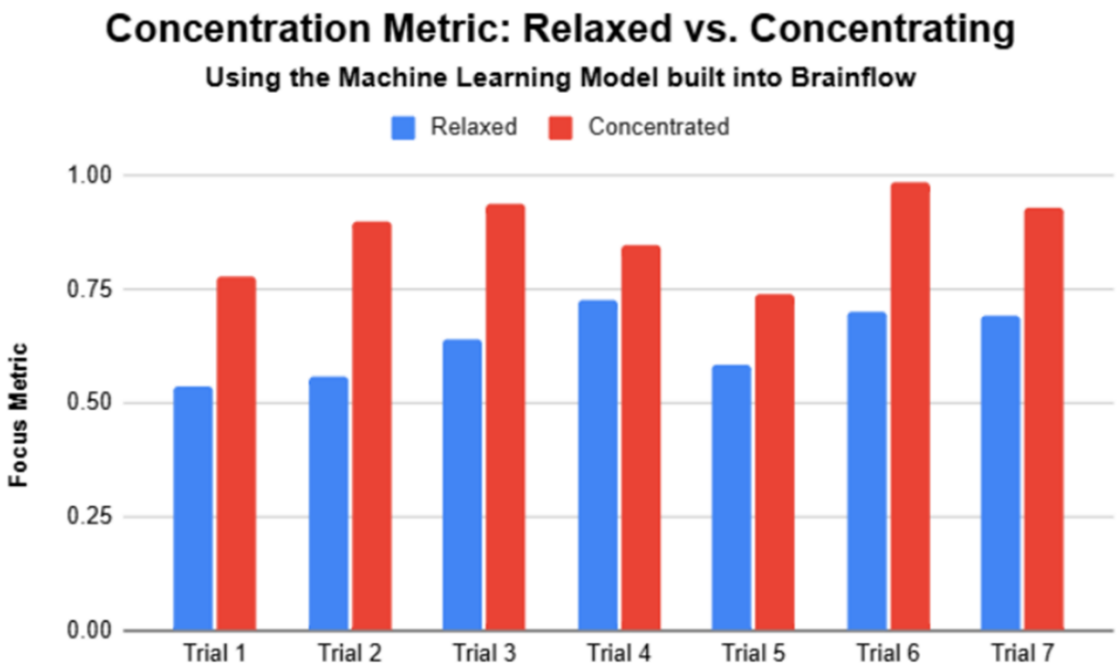


Figure 15: Concentration Metric Data

Commands

The controls work as follows: have the user close its eyes for approximately 10 seconds to have the drone take off. Afterwards the user can either double blink and concentrate to either move forward or can double blink and not concentrate to rotate.



Figure 16: Drone Controls

4. Implementations

4.1 Brain Controlled System

The prototype of our design meets all the requirements necessary. It is capable of taking off, hovering, landing, and even has directional capabilities. We use an electrode headband to secure snap and comb electrodes to our head to ensure valid signals. The prototype uses alpha waves, beta waves, and EOG spikes to function. When there are peaks in alpha waves, the drone will take off or land, evident by closed eyes. When there are two EEG spikes and beta waves' focus metric is either focused or relaxed, the drone will move forward or rotate respectively. The motion-tracking subsystem used a stereo-camera setup and a fine-tuned YOLOv8n model to detect the drone in real time. Stereo calibration, rectification, and disparity were then used to estimate the drone's approximate relative position.

There were many design decisions that our group went through to arrive at this prototype. The first issue we ran into was choosing how to secure the electrodes to our head in order to gain valid signals. Our first idea was to just tape the electrodes to our head. This idea was effective at gaining very rudimentary signals and to ensure that our CYTON Board was functioning properly. However, it was difficult to secure electrodes to portions of the head where there was hair. Therefore, we attempted to 3D print a design for a headset that could screw electrodes into place. After assembling our design, we learned that the rigidity of a headset was inefficient for our prototype as we needed to consistently move the electrode when switching between pilots for the drone. Furthermore, in order for the headset to be applicable for all users, it needed to be larger which led to loose connections. Our final design idea was inspired from our earlier community day involvement. We used wrist bands to secure flat head electrodes to children's arms so they could gain signals to control cars. To ensure this could be carried onto our brain controlled drone project, we created a barebone prototype from these wristbands. By cutting and sewing them together we created a sweatband that held the electrodes to place but still allowed for adjustability. With this discovery, we purchased a higher quality headband with holes that could fit our electrodes

Another design decision that we made was related to our controls. We wanted our prototype to function humanly, meaning that the controls were not overly complicated. One of our most reliable signals was eyes closed or eyes opened in correspondence with their alpha wave amplitudes. However, it is irrational to have our eyes closed when we are attempting to pilot the drone. Therefore, we chose this command to control our takeoff and landing. After all, we could hear when the drone took off and landed because of the propellers, giving a signal for us to open our eyes again. Next was the directional controls. The focus metric was incredibly difficult to control which made it unreliable as an action command. Therefore, we used it as a threshold for different controls. We chose double blinking as an action command because the spikes in EOG signals were consistent and blinking twice within 250ms could isolate purposeful commands. We now needed to decide what these commands would control. We tried having our drone fly back and front or left and right depending on focus metric being concentrated or relaxed but switching between front and back to left and right was inconsistent since our single blink switch would be recognized as double blinks too often. Therefore, we decided that focus would have our drone move forward and relaxed would have the drone rotate, preserving all the directions of our drone but less efficient.

4.2 Motion Tracking System

The motion-tracking prototype was implemented using a stereo-camera setup connected to the NVIDIA Jetson AGX Orin. Two e-CAM20_CUOAGX cameras were mounted in fixed positions and configured to capture synchronized left and right frames of the drone during operation. After the camera drivers and Jetson software environment were successfully configured, the system was used to acquire stereo images for real-time processing. Calibration and rectification were applied so that corresponding image points from the two cameras could be compared consistently. This made it possible to generate a disparity map and estimate the drone's approximate relative position with respect to the stereo camera frame.

To detect the drone reliably in each frame, a fine-tuned YOLOv8n object-detection model was used. Initial testing showed that the model performed well against simple backgrounds but was less reliable when the drone appeared in cluttered scenes or at larger distances. To improve robustness, additional images taken by the team in the actual testing and demo environments were labeled and added to the training dataset. These added images included the drone on the floor, on tables, near chairs, and in the final demonstration area under more realistic background conditions. After fine-tuning the model on this expanded dataset, drone detection became significantly more stable and no longer depended on placing a bright white background directly behind the drone.

Once the drone was detected in both camera views, the center of the detected bounding box was used as the image feature for position estimation. The disparity between the left and right detections was then used to estimate depth, and the output interface displayed the estimated drone coordinates together with the detection result and disparity visualization. During final testing, the detection portion of the motion-tracking subsystem operated reliably enough for demonstration and monitoring purposes. The main remaining limitation was not detection, but coordinate precision. For that reason, the final version of the interface emphasized stable detection and understandable position output while treating the reported coordinates as approximate rather than precise measurements.

5. Testing and Evaluation

Environmental Compatibility Requirements
[MESDP-BCD-01] The brain controlled drone shall fly within a confined space [D]
Status: Complete

This requirement was met through the use of a hard limit in the code for the drone control for how far it could travel. By defining an X and Y direction from takeoff, a counter increments or decrements based on direction and distance traveled. The drone can't move any further in the same direction once it hits an X or Y minimum or maximum. The code was tested with the drone and successfully prevented the drone from leaving a 120x120 cm square located around the take off point. The drone does not move up and down beyond the takeoff and land so no additional vertical limits were required.

[MESDP-BCD-02] The brain controlled drone shall not damage the surrounding environment through pollution or debris [I]
Status: Complete

This requirement was met through the same code parameters from MESDP-BCD-01, as well as through highlighting the confined space with tape so that it was clear the drone would not move beyond the boundaries and damage the surrounding area or anyone nearby. Throughout all testing and demonstration the drone did not damage anyone or anything.

Functional System Requirements
[MESDP-BCD-03] The brain controlled drone shall acquire, process, and display biopotential signal to extract user-intent features [A]

Status: Complete

This requirement was met through the use of the Cyton board, and the Brainflow Python library. One electrode was placed on the back of the head, four others were placed on the forehead and by the temples. The electrode on the back of the head was used to monitor the amplitude of alpha waves produced by the occipital lobe. When someone closes their eyes, an alpha wave amplitude spike can be detected here giving the first EEG signal. The other four electrodes were used to measure the level of concentration by analyzing the amplitudes of different frequency ranges (mainly beta waves) and feeding them into a machine learning algorithm built into the brainflow library. This ML model was designed to predict whether or not someone is concentrating, giving a second EEG signal. The alpha wave average was displayed in text and the output of the ML model (producing a number between 0-1 that moved up as the person concentrated more) was displayed on a graph.

[MESDP-BCD-04] The drone shall be able to takeoff, hover and land [D]
Status: Complete

This requirement was met with the DJI Tello Drone that was purchased. Through the djitellopy library, simple commands can be sent to the drone over a wifi signal to have it takeoff, hover, land, and much more. The takeoff or landing commands were sent if it was detected that the user closed their eyes for about 10 seconds.

[MESDP-BCD-05] The drone shall be programmable, wirelessly controlled, and rechargeable [T]
Status: Complete

This requirement was met with the DJI Tello Drone that was purchased. The DJI Tello could be programmed using python and the djitellopy library. The drone is wirelessly connected to a laptop running the code through a wifi connection, and the drone houses a battery that is rechargeable and replaceable

Maintenance System Requirements
[MESDP-BCD-06] The drone shall be reprogrammable and easy to reboot in the event of any malfunction in software [D]
Status: Complete

This requirement was met with the DJI Tello Drone. Reprogramming the drone simply required changing the code and sometimes pressing the power button on the drone again as it tended to disconnect and turn off after a few seconds without any new commands.

[MESDP-BCD-07] The drone shall be capable of operating, including repeated take offs, for at least 20 minutes before needing any batteries recharged or replaced [D]
Status: Complete

This requirement was met through the DJI Tello and additional batteries bought. The drone would receive about 13 minutes on a single battery of air time which was enough for 20 minutes

of operation given the ground time while calibrations were happening and EEG signals processed. Additional time could be met by simply taking out the battery and putting in a charged one.

Stretch Goals:
[MESDP-BCD-08] The brain controlled drone should move only on brain biopotential signal [D]
Status: Incomplete

This stretch goal was not met as EOG signals were also used in drone control. Though EMG was completely avoided in the final design.

[MESDP-BCD-09] The drone should be able to move directionally [D]
Status: Complete

This stretch goal was met through the commands being sent to the drone based on the concentration metric and EOG blink control. Whenever the user performed two blinks within half of a second, the drone would either rotate counter clockwise 90 degrees or move forward 30 cm based on whether the concentration metric was above or below threshold. This gave the drone the ability to move left, right, forward and backward based on which way it faced.

[MESDP-BCD-10] The drone should be tracked while moving at all speeds [I]
Status: Complete

This stretch goal was met through the NVIDIA Jetson AGX Orin motion-tracking subsystem using two stereo cameras and a fine-tuned YOLOv8n detection model. The system was able to detect and continuously track the drone in real time while it was moving during testing and demonstration. Early testing showed weaker performance in cluttered backgrounds and at larger distances, so additional labeled images from the lab and final demo environment were added to fine-tune the model. After retraining, tracking became much more reliable and no longer depended on using a plain white background behind the drone. While the coordinate output was approximate rather than highly precise, the system successfully met the project goal of visually tracking the drone as it moved.

[MESDP-BCD-11] The display that shows the bipotential signal and the position of the drone should act as a clean, easily understandable user interface [D]
Status: Complete

This stretch goal was met through the design of a user interface built with the Pyside6. The concentration level and threshold is displayed at the top, and the drones X,Y position is displayed at the bottom. Other important numbers such as the alpha wave average are displayed in the middle.

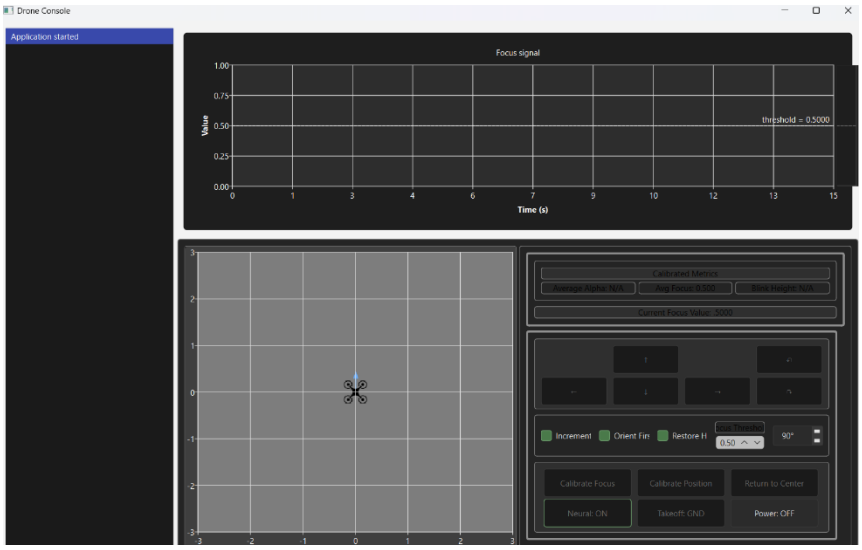


Figure 17: UI Interface

6. Schedule and Finances

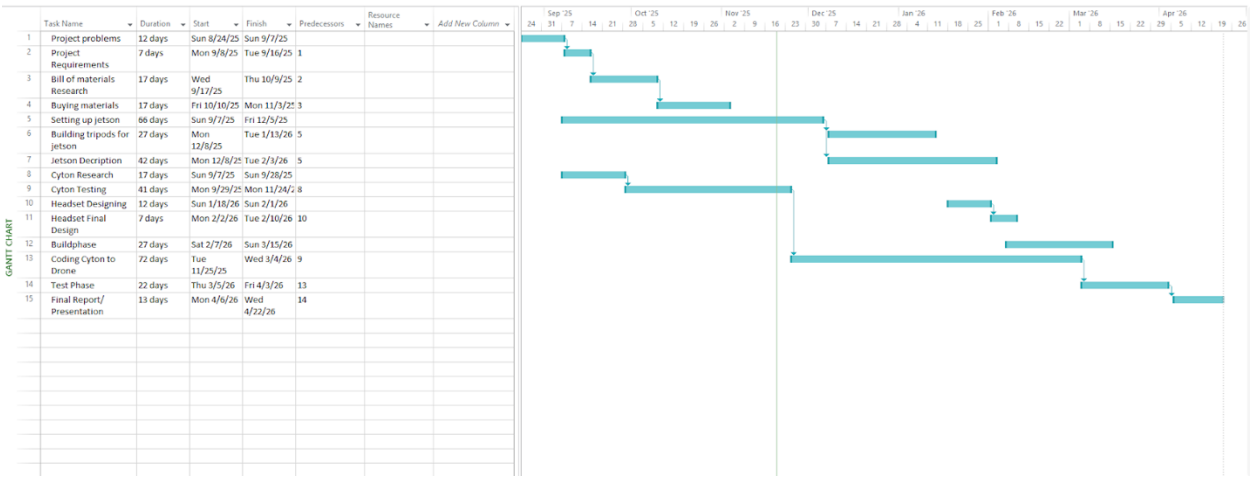
6.1 Finances

Item	Quantity	Cost
DJI Tello Drone	1	$274.99
DJI Battery Charger	2	$18.00
DJI Battery	6	$89.94
Leopard Imaging Micro Coax I-PEX Cable	3	$214.50
Open BCI Cyton Board	1	$1249.00
Open BCI EMG/EEG Snap Electrode Cables	10	$54.99
Open BCI EEG Snap Electrodes	5	$29.99
Kendell EMG ECG Foam Solid Gel Electrodes	50	$24.99
e-CAM_CUOAGX Cameras	2	Provided by Binghamton University
M6 Machining Screw	2	Provided by Binghamton University
Camera Holder	2	Manufactured by ETS Binghamton

AL2020 Aluminum Bar Extrusion	1	Provided by Binghamton University
NVIDIA Jetson Orin	1	Provided by Binghamton University

Total: $2136.40

6.2 Gantt Chart



	Task Name	Duration	Start	Finish	Predecessors	Resource Names
1	Project problems	12 days	Sun 8/24/25	Sun 9/7/25		
2	Project Requirements	7 days	Mon 9/8/25	Tue 9/16/25	1	
3	Bill of materials Research	17 days	Wed 9/17/25	Thu 10/9/25	2	
4	Buying materials	17 days	Fri 10/10/25	Mon 11/3/25	3	
5	Setting up jetson	66 days	Sun 9/7/25	Fri 12/5/25		
6	Building tripods for jetson	27 days	Mon 12/8/25	Tue 1/13/26	5	
7	Jetson Decription	42 days	Mon 12/8/25	Tue 2/3/26	5	
8	Cyton Research	17 days	Sun 9/7/25	Sun 9/28/25		
9	Cyton Testing	41 days	Mon 9/29/25	Mon 11/24/2	8	
10	Headset Designing	12 days	Sun 1/18/26	Sun 2/1/26		
11	Headset Final Design	7 days	Mon 2/2/26	Tue 2/10/26	10	
12	Buildphase	27 days	Sat 2/7/26	Sun 3/15/26		
13	Coding Cyton to Drone	72 days	Tue 11/25/25	Wed 3/4/26	9	
14	Test Phase	22 days	Thu 3/5/26	Fri 4/3/26	13	
15	Final Report/ Presentation	13 days	Mon 4/6/26	Wed 4/22/26	14	

GANTT CHART

Figure 18: Gantt Chart

7. Social Impact

The brain controlled drone was created for the purpose of helping others. There are many disabled and elderly people that are required to rely on others because of their condition. The brain controlled drone's research and implementation gives hope to the general public that they can be independent in the future without the limitations of their condition. Furthermore, the brain controlled drone revealed a lot about the effect of brain waves and signals on public health. After all, having increased beta waves leads to a more focused and healthy brain, something that is practiced when piloting the drone.

The brain controlled drone is also incredibly safe as safety was one of our teams top requirements. The chosen drone is small and has a lower set speed to ensure momentum and impulse does not damage the surrounding area. Furthermore, the drone has protective guards on its propellers to ensure that onlookers and the environment does not interfere with the flight of the drone. Brain waves are also incredibly difficult to control, so setting the action control to our most reliable signal helps mitigate any false inputs. Finally, if the drone begins to malfunction, we implement a kill switch to force land the drone.

Our system is also entirely electric. The drone runs on rechargeable batteries and the CYTON Board that reads all of our brain signals is also run on a rechargeable battery.

Therefore, the system is very environmentally friendly since it causes no pollution or excess waste to function.

8. Project Operation

8.1 How To Operate Drone Control System

Setting up Headband

Before anything else, the Cyton Board needs to be set up and the electrode headband needs to be fitted to the user's head. The headband, shown in figure 19, is a strap that holds the electrodes in position against the scalp during operation.

Five electrodes total go on the headband. Two flat electrodes sit at Fp1 and Fp2 these are the forehead positions shown in figure 12. Three comb electrodes go at T3, T4, and O1, shown in Figure 11. Earclip electrodes are also needed and clip onto A1 and A2 on either ear.



Figure 19: Christian Wearing Headband

Connecting Electrodes to the Cyton Board

Each electrode connects to a specific channel on the Cyton Board. The earclips at A1 and A2 connect to the BIAS and SRB pins BIAS acts as the ground reference while SRB helps filter noise out of the signal. O1 connects to Channel 1, Fp1 to Channel 2, and Fp2 to Channel 3. T3 and T4 can go on either Channel 4 or Channel 5.

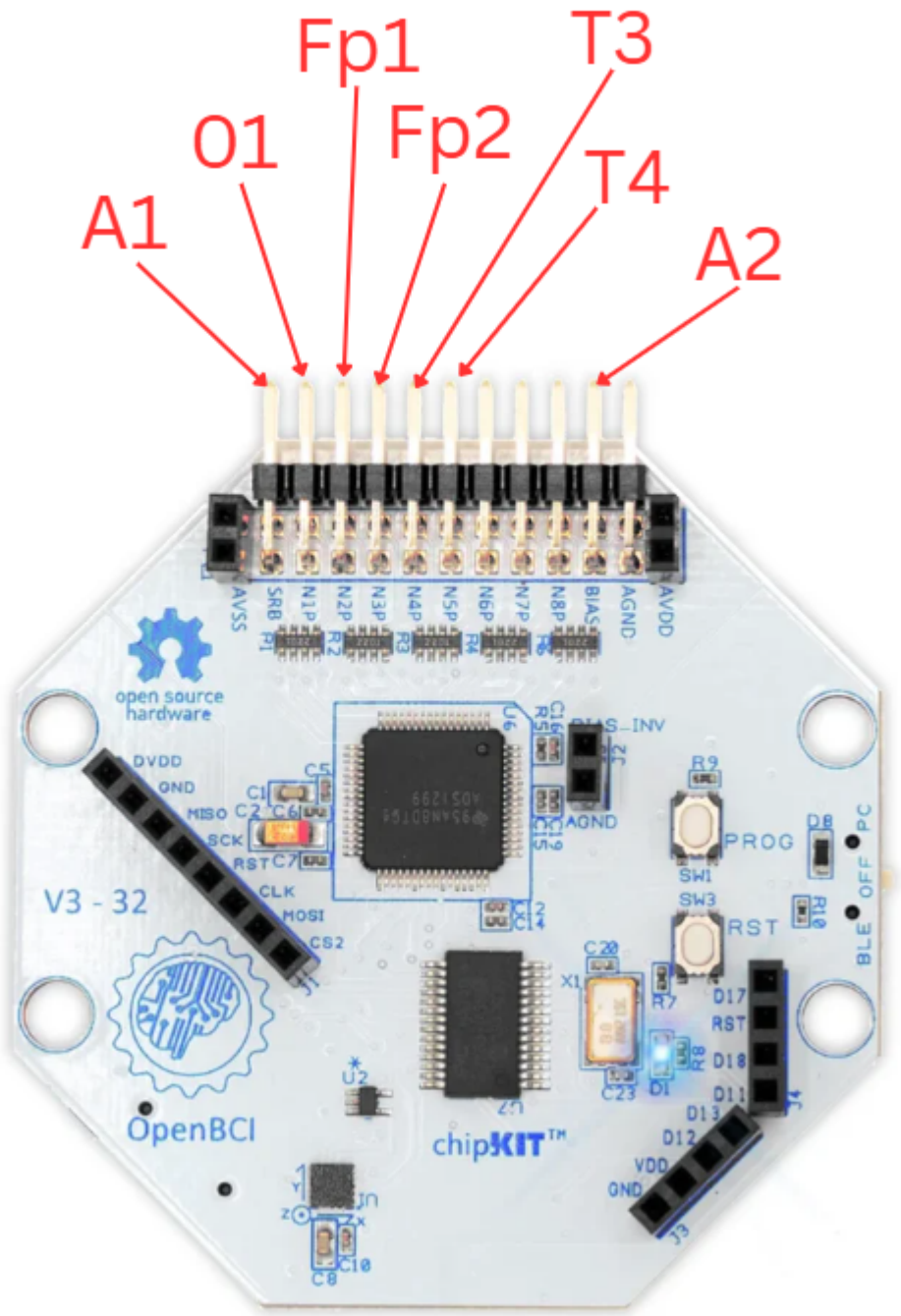


Figure 20: Cyton Board Electrode Connection (Bottom Layer)

Powering the System

Once the electrodes are wired up, connect the battery to the Cyton Board and power it on. The board should be secured in its protective casing and handled carefully. Battery life is long it only needs charging roughly once every two weeks. To charge it, use the dongle shown in figure 21 and connect it to the battery shown in figure 22.

With the board powered and secured, wrap the headband around the head. Then plug the Bluetooth dongle shown in Figure 14 into a USB port on the laptop this is what connects the Cyton Board to the laptop wirelessly. It is also recommended to download the OpenBCI GUI app, which lets you verify the electrode signals are coming through cleanly before running any flight code.

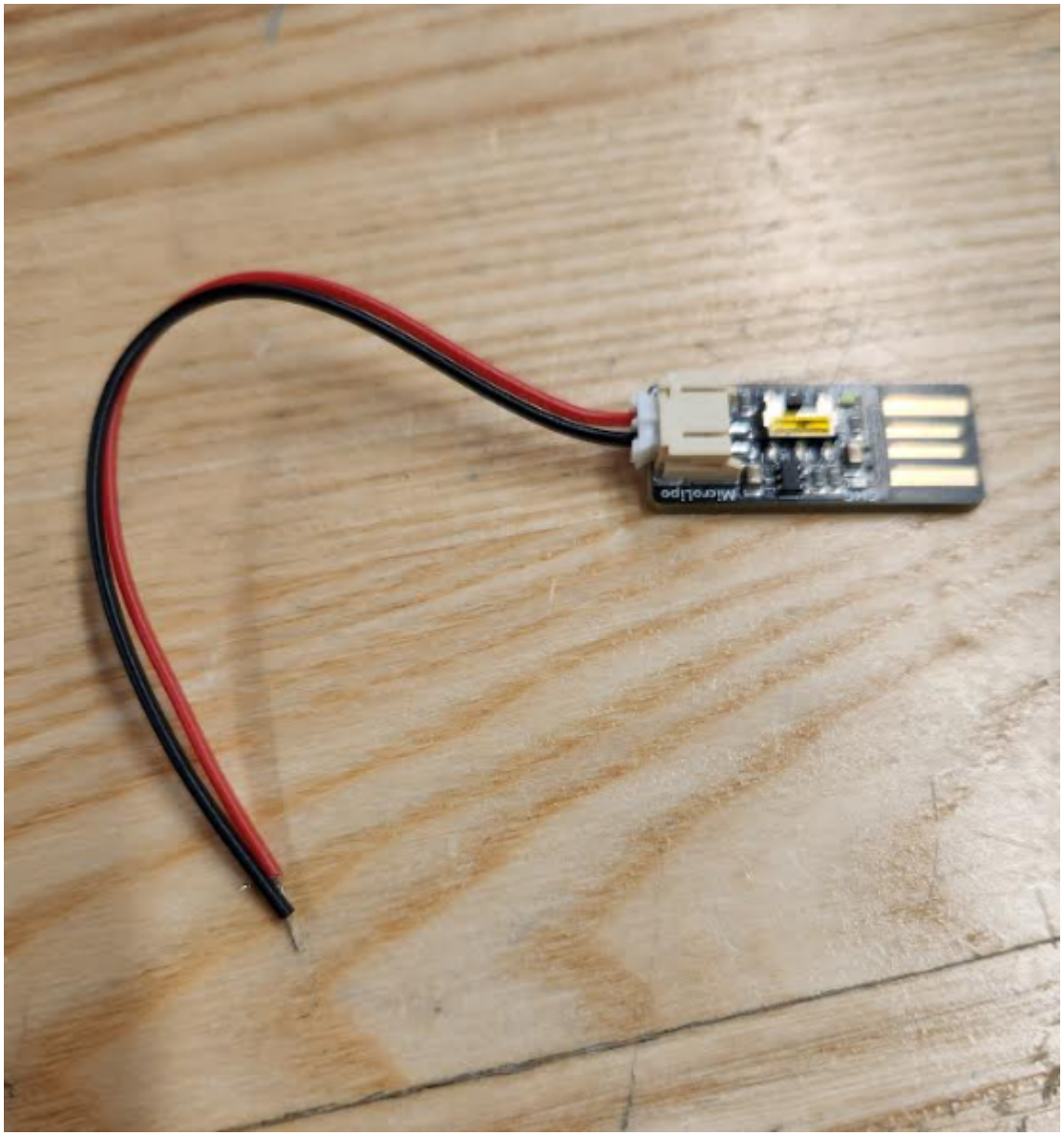


Figure 21: Battery Charger

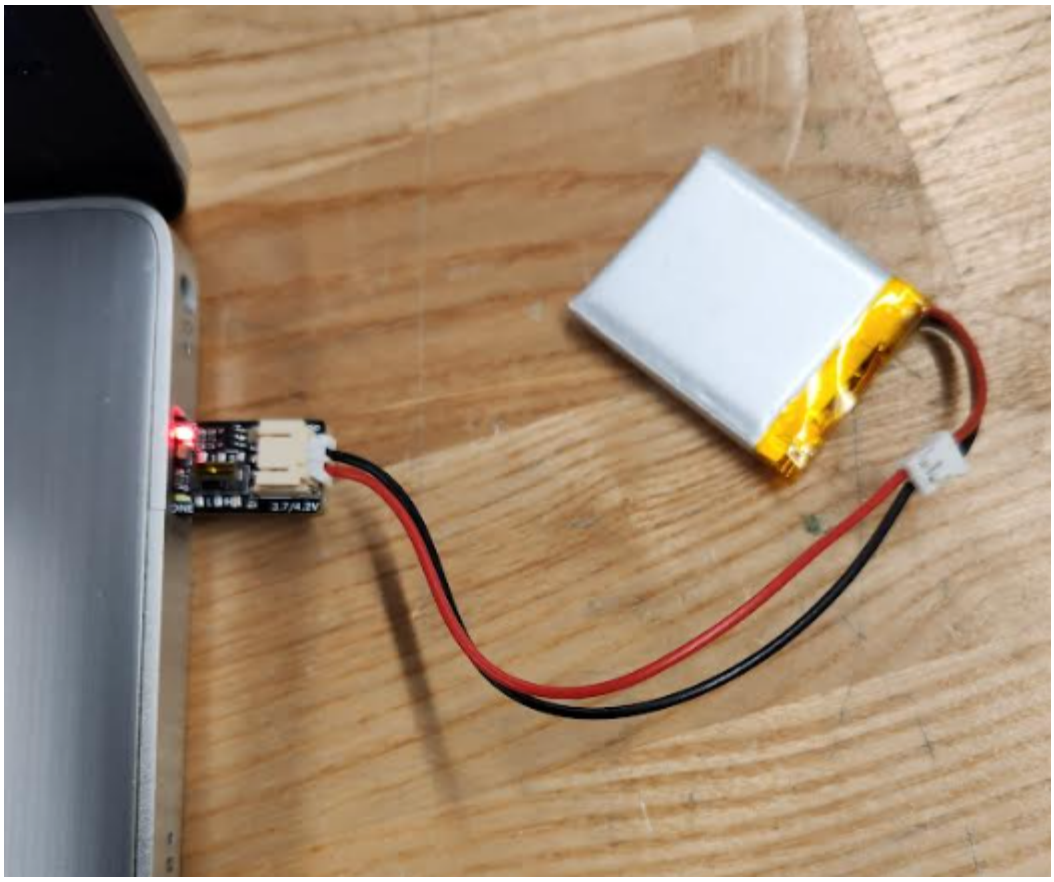


Figure 22: Battery Being Charged (Red light means charging)

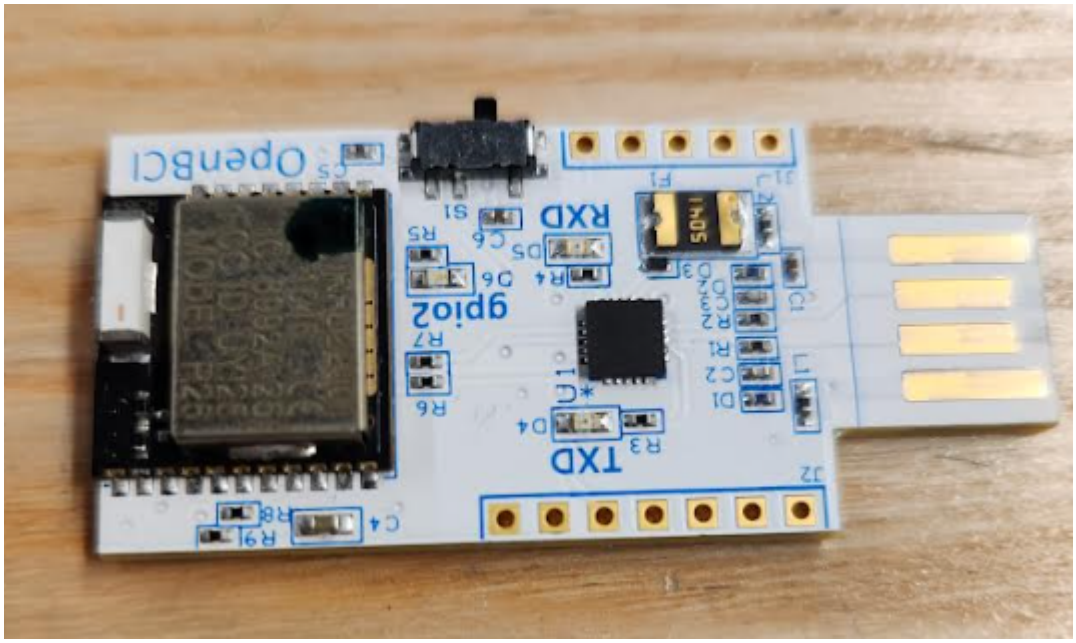


Figure 23: Open BCI Dongle

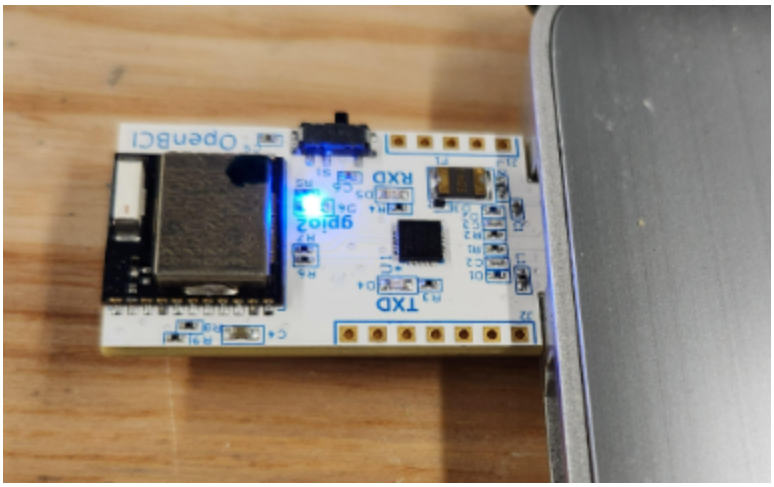


Figure 24: Dongle Connected to Laptop

Setting Up the Drone

Insert the battery into the drone and power it on. To connect it to the laptop, open the laptop's Wi-Fi settings and join the DJI Tello network. Once connected, the drone is ready to receive commands from the code.

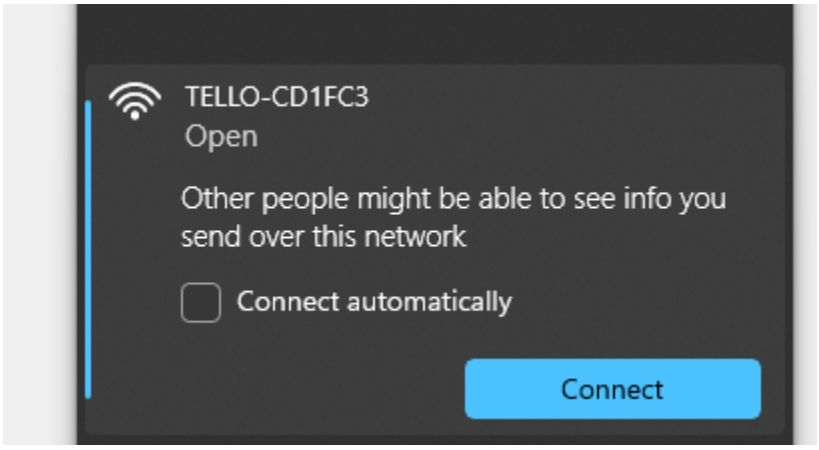


Figure 25: Connecting Drone to Laptop

Running the Code

The finished code is in Appendix B. PyCharm is the recommended environment to run it copy the code in, make two small adjustments: make sure the required imports are installed, and confirm the dongle is assigned to the correct USB port (this can also be changed directly in the code if needed).

There are two versions of the code, one with a UI and one without. The version without the UI runs better and produces more consistent results, so that's the one to use.

When the code starts, it takes a few seconds to initialize. After that, calibration begins automatically. During calibration, keep your eyes open and stay relaxed the system measures your baseline alpha and beta wave levels. Once calibration finishes, an alpha wave metric will appear on screen. A value below 10 means the signal quality is good and the session can continue. If the number comes in above 10, restart the code and run calibration again.

Once a clean calibration is confirmed, close your eyes to trigger the drone's takeoff sequence. From there, refer to Section 3.4 for the full flight controls.

8.2 How to Operate Motion Tracking System

To operate the motion-tracking subsystem, the NVIDIA Jetson AGX Orin must first be set up with all required hardware connections. The Jetson should be connected to its power supply, monitor, keyboard, and mouse. Both e-CAM20_CUOAGX embedded cameras must also be securely connected to the Jetson through the proper CSI interfaces before powering on the system. Once all connections are confirmed, the Jetson can be powered on and allowed to fully boot to the login screen.

After startup, the user should log into the Jetson desktop environment using the system credentials. Once logged in, a terminal window should be opened. The motion-tracking software is run through Docker so that all required dependencies, libraries, and configurations remain consistent. In the terminal, the user should first navigate to the Docker folder by entering:

```
cd ~/stereo_calibration/docker
```

After entering the Docker directory, the container should be launched with:

```
./run_docker.sh
```

This command starts the Docker environment used for the stereo vision and motion-tracking pipeline. Once the container is running, the user should navigate to the project directory if needed and then start the motion-tracking program by entering:

```
python3 motion_tracking_program.py
```

When executed successfully, the program opens the motion-tracking display window and begins reading from the stereo cameras in real time. The left side of the display shows the live drone detection and tracking results, while the right side shows the disparity map generated from the stereo pair. If the drone is detected, the system highlights it with a bounding box and reports its approximate position relative to the camera setup.

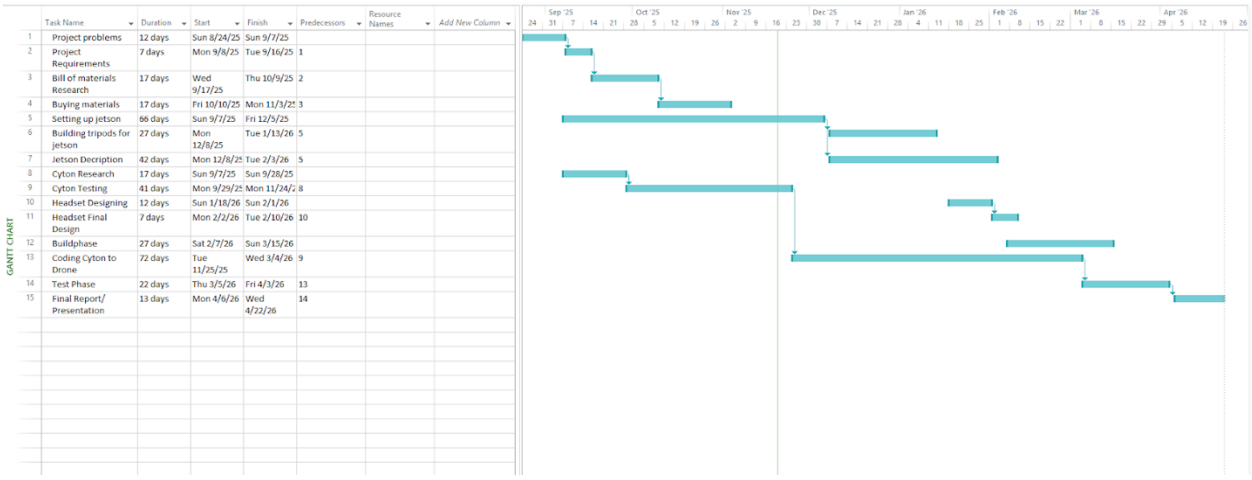
To stop the program, the user may hit the Q key on the keyboard to close out of the program. After use, the Docker container can be exited through closing the terminal window, and the Jetson should be shut down properly through the terminal or desktop power menu rather than being unplugged directly. Proper shutdown helps prevent file corruption and ensures the motion-tracking environment remains intact for future use. The following command should be entered to ensure proper shutdown:

```
sudo shutdown -h now
```

9. Bibliography

1. OpenBCI, *"Cyton Biosensing Board."* https://openbci.com/products/cyton-biosensing-board-8-channel , Accessed [November 2025].
2. Emotiv Inc., *"EPOC+ EEG System."* https://www.emotiv.com/epoc/ , Accessed [November 2025].
3. E. Fetz, "Operant Conditioning of Cortical Unit Activity," *Science*, vol. 163, no. 3870, pp. 955–958, 1969. https://www.science.org/doi/10.1126/science.163.3870.955 , Accessed [November 2025].
4. R. Nicolas-Alfonso and A. T. Bulling, "Using EEG Signals to Control a Quadcopter," *IEEE Transactions on Human-Machine Systems*, vol. 43, no. 2, pp. 130–138, 2013. https://ieeexplore.ieee.org/document/6471697/ , Accessed [November 2025].
5. NeuroSky Inc., *"MindWave Mobile."* https://store.neurosky.com , Accessed [November 2025].
6. g.tec Medical Engineering, *"g.Nautilus Wireless EEG System."* https://www.gtec.at/product/gnautilus-eeg/ , Accessed [November 2025].

Appendix A



	Task Name	Duration	Start	Finish	Predecessors	Resource Names	Add New Column
1	Project problems	12 days	Sun 8/24/25	Sun 9/7/25			
2	Project Requirements	7 days	Mon 9/8/25	Tue 9/16/25	1		
3	Bill of materials Research	17 days	Wed 9/17/25	Thu 10/9/25	2		
4	Buying materials	17 days	Fri 10/10/25	Mon 11/3/25	3		
5	Setting up jetson	66 days	Sun 9/7/25	Fri 12/5/25			
6	Building tripods for jetson	27 days	Mon 12/8/25	Tue 1/13/26	5		
7	Jetson Decription	42 days	Mon 12/8/25	Tue 2/3/26	5		
8	Cyton Research	17 days	Sun 9/7/25	Sun 9/28/25			
9	Cyton Testing	41 days	Mon 9/29/25	Mon 11/24/2	8		
10	Headset Designing	12 days	Sun 1/18/26	Sun 2/1/26			
11	Headset Final Design	7 days	Mon 2/2/26	Tue 2/10/26	10		
12	Buildphase	27 days	Sat 2/7/26	Sun 3/15/26			
13	Coding Cyton to Drone	72 days	Tue 11/25/25	Wed 3/4/26	9		
14	Test Phase	22 days	Thu 3/5/26	Fri 4/3/26	13		
15	Final Report/ Presentation	13 days	Mon 4/6/26	Wed 4/22/26	14		

GANTT CHART

Figure I: SDP 117 Gannt Chart

Appendix B

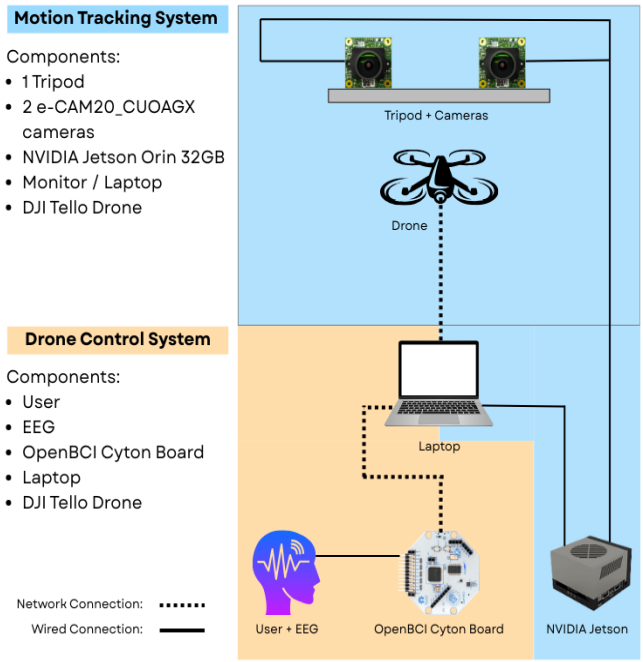


Figure II: Project Schematic

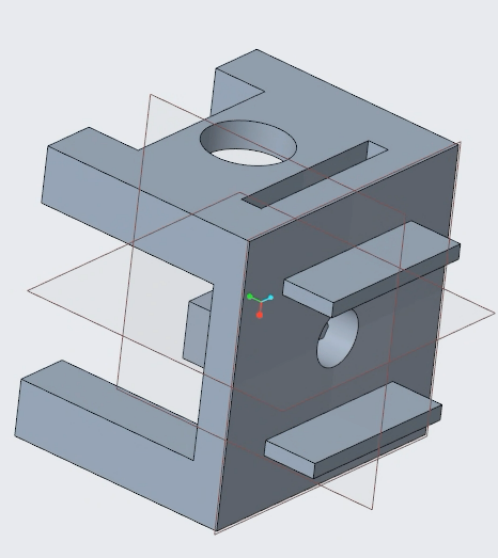


Figure III: Isometric View of Camera Tripod

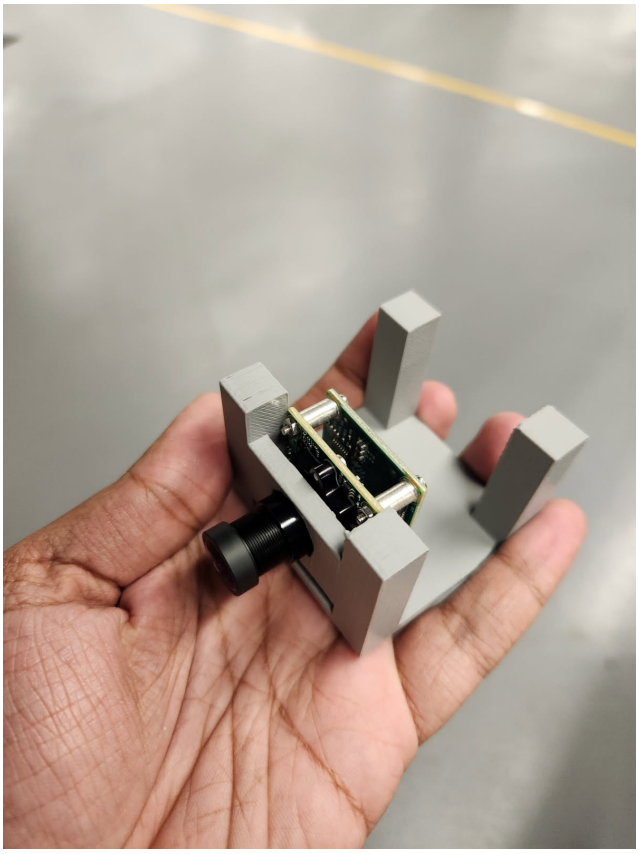


Figure IV: 3D Printed Camera Tripod with Camera inside

```
from brainflow.board_shim import BoardShim, BrainFlowInputParams, LogLevels, BoardIds
from brainflow.data_filter import DataFilter, WindowOperations, DetrendOperations
import brainflow.data_filter as df
from brainflow.ml_model import MLModel, BrainFlowMetrics, BrainFlowClassifiers,
BrainFlowModelParams
import time
import numpy as np
import scipy as sp
import csv
from djitellopy import Tello
import keyboard as kb

BoardShim.enable_dev_board_logger()
f_type = df.FilterTypes.BUTTERWORTH_ZERO_PHASE
params = BrainFlowInputParams()
board_id = 0
focus_data = np.zeros((24, 0))  #32 for simulated board, 24 for cyton
#focus_data = np.zeros((32, 0)) #Simulated Board
params.serial_port = 'COM7'
board_descr = BoardShim.get_board_descr(board_id)
sampling_rate = BoardShim.get_sampling_rate(board_id)
eeg_channels = BoardShim.get_eeg_channels(board_id)
board = BoardShim(board_id, params)
board.prepare_session()
board.config_board('x1050110X')
board.config_board('x2050110X')
board.start_stream()
BoardShim.log_message(LogLevels.LEVEL_INFO.value, 'Board stream started')
nfft = DataFilter.get_nearest_power_of_two(sampling_rate)

total_c1 = []
total_c2 = []
old_band_power_beta = 0
old_band_power_alpha = 0
EC_counter = 0

def send_to_UI(focus,average_focus,closed,blinking):
  #print(focus)
  #print(closed)
  #print(blinking)
  values = [focus[0], average_focus, closed, blinking]
  #print(values)
  with open('ui_data.csv', 'w') as csvfile:
    writer = csv.writer(csvfile,delimiter=',')
    for value in values:
      writer.writerow([value])
```

```
def calibrate_metrics(eyes_data,blink_data,concentration_data):
    print(len(eyes_data))
    print("calculating metrics...")
    recalibrate = False
    #EYES OPEN CALIBRATION----
    eyes_recent = eyes_data[-2500:]
    DataFilter.detrend(eyes_recent, DetrendOperations.LINEAR.value)
    psd1 = DataFilter.get_psd_welch(eyes_recent, nfft, nfft // 2, sampling_rate,
                                    WindowOperations.BLACKMAN_HARRIS.value)
    average_alpha_power = DataFilter.get_band_power(psd1, 7.0, 13.0)
    print(average_alpha_power)
    if average_alpha_power > 20:
        recalibrate = True
    #FOCUS CALIBRATION
    concentration_recent = (concentration_data[:,0:])
    concentration_recent = np.ascontiguousarray(concentration_recent)
    bands1, stddev1 = DataFilter.get_avg_band_powers(concentration_recent, [2, 3, 4, 5],
sampling_rate, True)
    feature_vector1 = bands1
    # print(feature_vector)
    concentration_average_array = mindfulness.predict(feature_vector1)
    concentration_average = concentration_average_array[0]
    #print(concentration_average)
    #BLINK CALibration
    f_type = df.FilterTypes.BUTTERWORTH_ZERO_PHASE
    df.DataFilter.perform_bandpass(blink_data, 250, 5, 50, 2, f_type, 0)
    blink_power = np.square(blink_data[500:-500])

    blinks, properties1 = sp.signal.find_peaks(blink_power, height=1000, distance=20)
    peak_amplitudes = properties1['peak_heights']
    print(peak_amplitudes)
    #blink =  max(peak_amplitudes)
    blink_metric  = 1000#0.7*blink
    return average_alpha_power, concentration_average, blink_metric, recalibrate



mindfulness_params =
BrainFlowModelParams(BrainFlowMetrics.MINDFULNESS.value,BrainFlowClassifiers.DEFA
ULT_CLASSIFIER.value)
mindfulness = MLModel(mindfulness_params)
mindfulness.prepare()
focus_total = []
eyes_closed = False
double_blink = False
```

```
flying  = False
focus_calibrated = False
keep_alive_sent = False
calibration_needed = True
initialize_printed = False
blink_printed = False
junk = board.get_board_data()
double_blink_time = 0
blink_time = 0
eyes_closed_time = 0
ct_control_buffer =0
counter_time = 0
direction = 0
position_x = 0
position_y = 0

tello = Tello()
tello.connect()
start_time = time.time()
while not (kb.is_pressed("space")):
  try:
      current_time = time.time() - start_time
      rounded_current_time = round(current_time)
      #collect data----------------------------------------
      data = board.get_board_data()
      c1_data = data[1]
      c2_data = data[2]
      total_c1 = np.append(total_c1, c1_data)
      total_c1_f = total_c1.copy()
      total_c2 = np.append(total_c2, c2_data)
      total_c2_f = total_c2.copy()
      if data.size > 0:
          focus_data = np.concatenate((focus_data, data), axis=1)
          eeg_channels = focus_data[1:9, :]

      if current_time < 10:
          if initialize_printed == False:
              print("INITIALIZING metrics --> RELAX!")
              initialize_printed = True
          if rounded_current_time != counter_time:
              print(rounded_current_time,'/10')
              counter_time = rounded_current_time
          continue
      elif calibration_needed == True:
          average_alpha, focus_ave, blink_height, calibration_needed
=calibrate_metrics(total_c1_f,total_c2_f,focus_data)
```

```
            print('alpha_power=',average_alpha, 'focus_ave=',focus_ave, 'blink_height=',blink_height)
            if focus_ave > .99:
                focus_ave = .99
            if calibration_needed == True:
                total_c1 = []
                total_c2 = []
                print('BAD DATA RECALIBRATING ')
                start_time = time.time()
                continue

        #-------------------------------------------------

        #ML STUFF-----------------------------------------------------------------
        if focus_data.shape[1] >= 2500:
            recent_data = (focus_data[:, -2500:])
            recent_data = np.ascontiguousarray(recent_data)
            bands, stddev = DataFilter.get_avg_band_powers(recent_data, [2,3,4,5], sampling_rate,
True)
            feature_vector = bands
            focus_merit = mindfulness.predict(feature_vector)
            focus_total.append(focus_merit)
        else:
            print("Initializing focus merit...")
            continue


        #DOUBLE BLINK STUFF-------------------------------------------------------------------------
------------------
        f_type = df.FilterTypes.BUTTERWORTH_ZERO_PHASE
        df.DataFilter.perform_bandpass(total_c2_f, 250, 5, 50, 2, f_type, 0)
        power_c2 = np.square(total_c2_f[-375:-125])
        #print(power_c2)
        if len(power_c2) > 0:
            blinks, properties  = sp.signal.find_peaks(power_c2, height = blink_height, distance =
20)  #changing 1500 to blink height

            if (len(blinks)>=2) and (current_time > blink_time+3):
                print('DETECTED DOUBLE BLINK', 'focus merit', focus_merit, '/', focus_ave)
                double_blink = True
                blink_time = current_time
            #elif (len(blinks) == 1) and (current_time > blink_time + 3):
                #print('DETECTED SINGLE BLINK, CHANGING DIRECTION')
                #if direction ==0:
                    #direction = 1
                    #print('MOVING LEFT AND RIGHT')
                #else:
```

```
                #direction =0
                #print('MOVING FORWARD AND BACK')
                #print(properties['peak_heights'])


    #EYES OPEN and Closed----------------------------------------------------------------------------
-------
    if len(total_c1_f) > 750:
        total_c1_f_recent = total_c1_f[-750:]
        DataFilter.detrend(total_c1_f_recent, DetrendOperations.LINEAR.value)
        psd = DataFilter.get_psd_welch(total_c1_f_recent, nfft, nfft // 2, sampling_rate,
                                       WindowOperations.BLACKMAN_HARRIS.value)
        band_power_alpha = DataFilter.get_band_power(psd, 7.0, 13.0) #this is for alpha

        if round(band_power_alpha,4) != round(old_band_power_alpha,4):
            #print(average_alpha, '   ', band_power_alpha, '   ', (band_power_alpha - average_alpha)
/ average_alpha)
            old_band_power_alpha = band_power_alpha
            if ((band_power_alpha-average_alpha)/average_alpha > 1) and (band_power_alpha <
100):   # CHANGED TO 1.5 from 1
                EC_counter += 1
                print(EC_counter,'-->',band_power_alpha)
                if EC_counter >= 5:
                    print('EYES CLOSED')
                    eyes_closed = True
                    EC_counter = 0
            #elif ((band_power_alpha-average_alpha)/average_alpha < 1) or (band_power_alpha
<30): EC_counter = 0
            else: EC_counter = 0

    #-------------------------------------------------------------------------------------------------
    #UI CONTROL
    if eyes_closed:
        eyes_closed_ui = 1
    else:
        eyes_closed_ui = 0

    if double_blink:
        double_blink_ui = 1
    else:
        double_blink_ui = 0

    send_to_UI(focus_merit,focus_ave,eyes_closed_ui,double_blink_ui)

    # DRONE CONTROL ------------------------------------------------
    # Take off / Land-------
```

```
    if (eyes_closed == True) and (current_time > (eyes_closed_time + 8)) and (
            current_time > (double_blink_time + 5)):
        if flying == False:
            print('EYES CLOSED: SENDING TAKEOFF CAMMAND')
            tello.takeoff()
            print('FLYING')
            flying = True
            eyes_closed = False
            eyes_closed_time = current_time
        if flying == True and (current_time > eyes_closed_time + 5):
            print('EYES CLOSED: SENDING LAND CAMMAND, PROGRAM SHUTTING
DOWN')
            flying = False
            eyes_closed = False
            eyes_closed_time = current_time
            break
    else:
        eyes_closed = False

    # rotate 90 ---------------------
    if (double_blink == True) and (focus_merit <= focus_ave) and (flying == True) and (
            current_time > (double_blink_time + 5)):
        print('RELAXED DOUBLE BLINK: ROTATING 90 DEGREES', '---> focus merit:',
focus_merit)
        tello.rotate_counter_clockwise(90)
        if direction < 3:
            direction += 1
        elif direction == 3:
            direction = 0
        double_blink = False
        double_blink_time = current_time

    # Safeguards ----------------------------------
    if (position_x == 2) and (direction == 3):
        can_move = False
        if double_blink == True:
            print('+x limit reached')
    elif (position_y == 2) and (direction == 0) and (double_blink == True):
        can_move = False
        if double_blink == True:
            print('+y limit reached')
    elif (position_x == -2) and (direction == 1) and (double_blink == True):
        can_move = False
        if double_blink == True:
            print('-x limit reached')
    elif (position_y == -2) and (direction == 2) and (double_blink == True):
```

```
            can_move = False
            if double_blink == True:
                print('-y limit reached')
        else:
            can_move = True

        # Move forword --------------------
        if (double_blink == True) and (focus_merit > focus_ave) and (flying == True) and
(can_move == True):
            print('FOCUSED DOUBLE BLINK: MOVING FORWARD 30 CM', '---> focus merit:',
focus_merit)
            tello.move_forward(30)
            if direction == 0:
                position_y += 1
            elif direction == 1:
                position_x -= 1
            elif direction == 2:
                position_y -= 1
            elif direction == 3:
                position_x += 1

            double_blink = False

        #-----------------------------------------------------------------------------------------------------

        #MANUAL CHANGE OF CONCENTRATION THRESHOLD
        if current_time > ct_control_buffer + .1:
            ct_control_buffer = current_time
            if kb.is_pressed('up'):
                print('raising concentration threshhold')
                if focus_ave < .99:
                    focus_ave = focus_ave+.02
                else: focus_ave = .99
                print('new threshold:',focus_ave)
            elif kb.is_pressed('down'):
                print('lowering concentration threshhold')
                if focus_ave > .01:
                    focus_ave = focus_ave-.02
                else: focus_ave = .01
                print('new threshold:',focus_ave)

        #keep drone alive
        if (flying == True) and (round(current_time) % 5 ==0) and (keep_alive_sent == False):
            tello.send_command_without_return('command')
            print('keep_alive_sent')
            keep_alive_sent = True
```

```
        elif round(current_time) % 5 != 0: keep_alive_sent = False
    except:
        print('possible error')
        break

tello.land()
board.stop_stream()
board.release_session()
```

Figure 6: Drone Code (Run code on PyCharm)

MOTION TRACKING PROGRAM CODE:

```
# Drone Detection with YOLO
# Written By: Bhavi Patel
# Date: May 1, 2026
# Notes:
# Updated text output.
# Implements 4th iteration of YOLO model
# that uses images from our final demo
# location.

import cv2
import numpy as np
import time
import threading
import os
from collections import deque

from ultralytics import YOLO

# ==========================
# SETTINGS
# ==========================
# Camera capture resolution
WIDTH, HEIGHT = 1920, 1080

# Processing resolution (keep v2 speed)
PROCESS_W, PROCESS_H = 960, 540

# Display resolution
DISPLAY_W, DISPLAY_H = 1280, 720

LEFT_DEV = 0
RIGHT_DEV = 1

CALIB_FILE = "stereo_calib_sb_9x6_24mm.npz"
```

```python
MODEL_PATH = "models/drone_demo_finetune_best.pt"

STALE_TIMEOUT = 1.0

# ---------------------------
# Stereo disparity settings
# ---------------------------
MIN_DISPARITY = 0
NUM_DISPARITIES = 16 * 6
BLOCK_SIZE = 9

# Optional disparity improvements
USE_CLAHE = True
ENABLE_WLS = True
WLS_LAMBDA = 8000.0
WLS_SIGMA = 1.5

# ---------------------------
# Detection settings
# ---------------------------
CONF_THRESH = 0.20
DETECT_EVERY_N_FRAMES = 1
BOX_HOLD_FRAMES = 2
TARGET_CLASS_ID = 0
BOX_SMOOTH_ALPHA = 0.85   # 1.0 = no smoothing, 0.85 = responsive

# YOLO settings
YOLO_IMGSZ = 800
USE_HALF = True

# Display settings
MIRROR_DISPLAY = True
WINDOW_NAME = "SDP 117: Stereo YOLO Motion Tracking"
FOOTER_TEXT = "Left: Drone Detection and Tracking | Right: Disparity Map"
SHOW_DEBUG_DEPTH_TEXT = False

# ---------------------------
# Depth / coordinate settings
# ---------------------------
MM_TO_IN = 1.0 / 25.4
IN_TO_FT = 1.0 / 12.0

def round_to_half_foot(v_in):
    v_ft = v_in * IN_TO_FT
    return round(v_ft * 2.0) / 2.0

# Inner ROI inside the bbox for depth sampling
BODY_ROI_W_FRAC = 0.30
```

```
BODY_ROI_H_FRAC = 0.35
MIN_BODY_ROI_HALF_W = 8
MIN_BODY_ROI_HALF_H = 8
MAX_BODY_ROI_HALF_W = 20
MAX_BODY_ROI_HALF_H = 24

MIN_VALID_DISP = 2.0
MIN_VALID_DISP_PIXELS = 30
TRIM_PERCENT_LOW = 20
TRIM_PERCENT_HIGH = 80

# Coordinate smoothing
XYZ_HISTORY_LEN = 5

# ============================
# GSTREAMER PIPELINE
# ============================
def gstreamer_pipeline(device: int) -> str:
    return (
        f"v4l2src device=/dev/video{device} ! "
        f"video/x-raw,format=UYVY,width={WIDTH},height={HEIGHT} ! "
        "videoconvert ! "
        "video/x-raw,format=BGR ! "
        "appsink max-buffers=1 drop=true sync=false"
    )

# ============================
# CAMERA THREAD
# ============================
class LatestFrameGrabber:
    def __init__(self, dev_index: int):
        self.dev_index = dev_index
        self.cap = None
        self.lock = threading.Lock()
        self.frame = None
        self.ok = False
        self.stop_flag = False
        self.thread = None
        self.last_update_time = 0.0

    def open(self):
        self.cap = cv2.VideoCapture(gstreamer_pipeline(self.dev_index),
cv2.CAP_GSTREAMER)
        return self.cap.isOpened()

    def start(self):
        self.stop_flag = False
        self.thread = threading.Thread(target=self._run, daemon=True)
```

```
        self.thread.start()

    def _run(self):
        while not self.stop_flag:
            if self.cap is None or not self.cap.isOpened():
                time.sleep(0.05)
                continue

            ret, frm = self.cap.read()
            with self.lock:
                self.ok = bool(ret) and frm is not None
                if self.ok:
                    self.frame = frm
                    self.last_update_time = time.time()
            time.sleep(0.001)

    def get(self):
        with self.lock:
            frame_copy = None if self.frame is None else self.frame.copy()
            return self.ok, frame_copy, self.last_update_time

    def reopen(self):
        try:
            if self.cap is not None:
                self.cap.release()
        except Exception:
            pass
        time.sleep(0.25)
        return self.open()

    def stop(self):
        self.stop_flag = True
        try:
            if self.thread is not None:
                self.thread.join(timeout=1.0)
        except Exception:
            pass
        try:
            if self.cap is not None:
                self.cap.release()
        except Exception:
            pass

# ==========================
# HELPERS
# ==========================
def smooth_box(prev_box, new_box, alpha):
    if prev_box is None:
```

```
        return [float(v) for v in new_box]
    return [
        (1.0 - alpha) * prev_box[0] + alpha * new_box[0],
        (1.0 - alpha) * prev_box[1] + alpha * new_box[1],
        (1.0 - alpha) * prev_box[2] + alpha * new_box[2],
        (1.0 - alpha) * prev_box[3] + alpha * new_box[3],
    ]

def choose_best_detection(result_boxes, target_class_id=None):
    best = None
    best_conf = -1.0
    best_cls = None

    for b in result_boxes:
        conf = float(b.conf[0].item())
        cls_id = int(b.cls[0].item()) if b.cls is not None else -1

        if target_class_id is not None and cls_id != target_class_id:
            continue

        if conf > best_conf:
            xyxy = b.xyxy[0].cpu().numpy().tolist()
            best = xyxy
            best_conf = conf
            best_cls = cls_id

    if best is None:
        return None, None, None
    return best, best_conf, best_cls

def clamp_box(box, w, h):
    x1, y1, x2, y2 = [int(round(v)) for v in box]
    x1 = max(0, min(w - 1, x1))
    x2 = max(0, min(w - 1, x2))
    y1 = max(0, min(h - 1, y1))
    y2 = max(0, min(h - 1, y2))
    return x1, y1, x2, y2

def mirror_box_coords(x1, y1, x2, y2, img_w):
    mx1 = (img_w - 1) - x2
    mx2 = (img_w - 1) - x1
    return mx1, y1, mx2, y2

def mirror_x(x, img_w):
    return (img_w - 1) - x

def get_class_name(class_names, cls_id):
    if cls_id is None:
```

```
        return "Object"
    if isinstance(class_names, dict):
        return class_names.get(cls_id, str(cls_id))
    if isinstance(class_names, list):
        if 0 <= cls_id < len(class_names):
            return class_names[cls_id]
    return str(cls_id)

def median_or_none(dq):
    if len(dq) == 0:
        return None
    return float(np.median(np.array(dq, dtype=np.float32)))

def compute_body_roi(cx, cy, x1, y1, x2, y2, img_w, img_h):
    bw = x2 - x1
    bh = y2 - y1

    half_w = max(MIN_BODY_ROI_HALF_W, min(int(round(bw * BODY_ROI_W_FRAC * 0.5)),
MAX_BODY_ROI_HALF_W))
    half_h = max(MIN_BODY_ROI_HALF_H, min(int(round(bh * BODY_ROI_H_FRAC * 0.5)),
MAX_BODY_ROI_HALF_H))

    rx1 = max(0, cx - half_w)
    rx2 = min(img_w, cx + half_w)
    ry1 = max(0, cy - half_h)
    ry2 = min(img_h, cy + half_h)

    return rx1, ry1, rx2, ry2

# ==========================
# MAIN
# ==========================
def main():
    cv2.setUseOptimized(True)
    cv2.setNumThreads(2)

    if not os.path.exists(CALIB_FILE):
        print(f"ERROR: Calibration file not found: {CALIB_FILE}")
        return

    if not os.path.exists(MODEL_PATH):
        print(f"ERROR: YOLO model not found: {MODEL_PATH}")
        return

    # Load calibration
    data = np.load(CALIB_FILE)
    mapLx, mapLy = data["mapLx"], data["mapLy"]
    mapRx, mapRy = data["mapRx"], data["mapRy"]
```

```
    P1 = data["P1"]
    P2 = data["P2"]

    # Full-resolution intrinsics
    fx = float(P1[0, 0])
    fy = float(P1[1, 1])
    cx0 = float(P1[0, 2])
    cy0 = float(P1[1, 2])
    baseline_mm = abs(float(P2[0, 3]) / fx)

    # Scale intrinsics to processing resolution
    sx = PROCESS_W / WIDTH
    sy = PROCESS_H / HEIGHT
    fx_p = fx * sx
    fy_p = fy * sy
    cx0_p = cx0 * sx
    cy0_p = cy0 * sy

    print(f"Using fx(full): {fx:.2f} px")
    print(f"Using fy(full): {fy:.2f} px")
    print(f"Using cx0(full): {cx0:.2f} px")
    print(f"Using cy0(full): {cy0:.2f} px")
    print(f"Using baseline: {baseline_mm:.2f} mm")
    print(f"Processing at: {PROCESS_W}x{PROCESS_H}")

    # Load YOLO model
    model = YOLO(MODEL_PATH)
    class_names = model.names

    # Stereo matcher
    stereo_left = cv2.StereoSGBM_create(
        minDisparity=MIN_DISPARITY,
        numDisparities=NUM_DISPARITIES,
        blockSize=BLOCK_SIZE,
        P1=8 * BLOCK_SIZE * BLOCK_SIZE,
        P2=32 * BLOCK_SIZE * BLOCK_SIZE,
        disp12MaxDiff=1,
        uniquenessRatio=12,
        speckleWindowSize=100,
        speckleRange=2,
        preFilterCap=31,
        mode=cv2.STEREO_SGBM_MODE_SGBM_3WAY
    )

    stereo_right = None
    wls_filter = None
    use_wls = False
```

```
    if ENABLE_WLS and hasattr(cv2, "ximgproc") and hasattr(cv2.ximgproc,
"createRightMatcher"):
        try:
            stereo_right = cv2.ximgproc.createRightMatcher(stereo_left)
            wls_filter =
cv2.ximgproc.createDisparityWLSFilter(matcher_left=stereo_left)
            wls_filter.setLambda(WLS_LAMBDA)
            wls_filter.setSigmaColor(WLS_SIGMA)
            use_wls = True
            print("Using WLS disparity filtering")
        except Exception as e:
            print(f"WLS unavailable, using raw SGBM disparity: {e}")
            use_wls = False
    else:
        print("WLS not available, using raw SGBM disparity")

    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8)) if USE_CLAHE else None

    left = LatestFrameGrabber(LEFT_DEV)
    right = LatestFrameGrabber(RIGHT_DEV)

    if not left.open():
        print("ERROR: Left camera failed to open")
        return
    if not right.open():
        print("ERROR: Right camera failed to open")
        return

    left.start()
    right.start()

    cv2.namedWindow(WINDOW_NAME, cv2.WINDOW_NORMAL)
    cv2.resizeWindow(WINDOW_NAME, DISPLAY_W, DISPLAY_H)

    print("\nControls:")
    print("  q -> quit")
    print("  r -> reopen cameras\n")

    frame_idx = 0
    tracked_box = None
    tracked_conf = None
    tracked_cls = None
    hold_count = 0

    x_hist = deque(maxlen=XYZ_HISTORY_LEN)
    y_hist = deque(maxlen=XYZ_HISTORY_LEN)
    z_hist = deque(maxlen=XYZ_HISTORY_LEN)
```

```
    fps_ema = 0.0
    prev_loop_time = time.time()

    while True:
        okL, frameL, tL = left.get()
        okR, frameR, tR = right.get()
        now = time.time()

        # FPS estimate
        dt = max(now - prev_loop_time, 1e-6)
        inst_fps = 1.0 / dt
        fps_ema = inst_fps if fps_ema == 0.0 else 0.9 * fps_ema + 0.1 * inst_fps
        prev_loop_time = now

        if okL and (now - tL > STALE_TIMEOUT):
            print("Left camera stream stale -> reopening left camera...")
            left.reopen()
            time.sleep(0.2)

        if okR and (now - tR > STALE_TIMEOUT):
            print("Right camera stream stale -> reopening right camera...")
            right.reopen()
            time.sleep(0.2)

        if not okL or not okR:
            key = cv2.waitKey(10) & 0xFF
            if key == ord('q'):
                break
            if key == ord('r'):
                print("Reopening cameras...")
                left.reopen()
                right.reopen()
            continue

        # Rectify at full res
        rectL_full = cv2.remap(frameL, mapLx, mapLy, cv2.INTER_LINEAR)
        rectR_full = cv2.remap(frameR, mapRx, mapRy, cv2.INTER_LINEAR)

        # Downscale for processing
        procL = cv2.resize(rectL_full, (PROCESS_W, PROCESS_H),
interpolation=cv2.INTER_AREA)
        procR = cv2.resize(rectR_full, (PROCESS_W, PROCESS_H),
interpolation=cv2.INTER_AREA)

        # Disparity on unflipped processing frames
        grayL = cv2.cvtColor(procL, cv2.COLOR_BGR2GRAY)
        grayR = cv2.cvtColor(procR, cv2.COLOR_BGR2GRAY)
```

```
        if clahe is not None:
            grayL = clahe.apply(grayL)
            grayR = clahe.apply(grayR)

        dispL_raw = stereo_left.compute(grayL, grayR)

        if use_wls:
            dispR_raw = stereo_right.compute(grayR, grayL)
            disp_filtered = wls_filter.filter(dispL_raw, procL, None, dispR_raw)
            disparity = disp_filtered.astype(np.float32) / 16.0
        else:
            disparity = dispL_raw.astype(np.float32) / 16.0

        disparity[disparity < MIN_VALID_DISP] = 0

        # -------- YOLO DETECTION ON LEFT PROCESSING FRAME --------
        frame_idx += 1

        if frame_idx % DETECT_EVERY_N_FRAMES == 0:
            classes_arg = [TARGET_CLASS_ID] if TARGET_CLASS_ID is not None else None
            results = model.predict(
                source=procL,
                conf=CONF_THRESH,
                imgsz=YOLO_IMGSZ,
                device=0,
                half=USE_HALF,
                classes=classes_arg,
                verbose=False
            )

            if results and len(results) > 0:
                boxes = results[0].boxes
                best_box, best_conf, best_cls = choose_best_detection(boxes,
TARGET_CLASS_ID)

                if best_box is not None:
                    tracked_box = smooth_box(tracked_box, best_box, BOX_SMOOTH_ALPHA)
                    tracked_conf = best_conf
                    tracked_cls = best_cls
                    hold_count = BOX_HOLD_FRAMES
                else:
                    if hold_count > 0:
                        hold_count -= 1
                    else:
                        tracked_box = None
                        tracked_conf = None
                        tracked_cls = None
                        x_hist.clear()
```

```
                        y_hist.clear()
                        z_hist.clear()
            else:
                if hold_count > 0:
                    hold_count -= 1
                else:
                    tracked_box = None
                    tracked_conf = None
                    tracked_cls = None
                    x_hist.clear()
                    y_hist.clear()
                    z_hist.clear()
        else:
            if tracked_box is not None and hold_count > 0:
                hold_count -= 1

        info_lines = ["No drone detected"]
        xyz_text_terminal = None

        # Disparity visualization
        disp_vis = disparity.copy()
        disp_vis = cv2.normalize(disp_vis, None, 0, 255, cv2.NORM_MINMAX)
        disp_vis = np.uint8(disp_vis)
        disp_vis = cv2.applyColorMap(disp_vis, cv2.COLORMAP_JET)

        # Display images
        if MIRROR_DISPLAY:
            viewL = cv2.flip(procL.copy(), 1)
            viewDisp = cv2.flip(disp_vis.copy(), 1)
        else:
            viewL = procL.copy()
            viewDisp = disp_vis.copy()

        if tracked_box is not None:
            x1, y1, x2, y2 = clamp_box(tracked_box, PROCESS_W, PROCESS_H)

            if x2 > x1 and y2 > y1:
                cx = (x1 + x2) // 2
                cy = (y1 + y2) // 2

                # Use bbox-relative inner ROI instead of fixed center window
                rx1, ry1, rx2, ry2 = compute_body_roi(cx, cy, x1, y1, x2, y2,
disparity.shape[1], disparity.shape[0])

                roi_disp = disparity[ry1:ry2, rx1:rx2]
                valid_disp = roi_disp[roi_disp > MIN_VALID_DISP]

                if MIRROR_DISPLAY:
```

```
                    dx1, dy1, dx2, dy2 = mirror_box_coords(x1, y1, x2, y2, PROCESS_W)
                    dcx = mirror_x(cx, PROCESS_W)
                    dcy = cy
                    drx1, dry1, drx2, dry2 = mirror_box_coords(rx1, ry1, rx2, ry2,
PROCESS_W)
                else:
                    dx1, dy1, dx2, dy2 = x1, y1, x2, y2
                    dcx, dcy = cx, cy
                    drx1, dry1, drx2, dry2 = rx1, ry1, rx2, ry2

                # Draw overlays on left image
                cv2.rectangle(viewL, (dx1, dy1), (dx2, dy2), (0, 255, 0), 2)
                cv2.circle(viewL, (dcx, dcy), 5, (0, 0, 255), -1)
                cv2.rectangle(viewL, (drx1, dry1), (drx2, dry2), (0, 255, 255), 2)

                # Also draw depth ROI on disparity panel
                cv2.rectangle(viewDisp, (drx1, dry1), (drx2, dry2), (0, 255, 255), 2)
                cv2.circle(viewDisp, (dcx, dcy), 5, (0, 0, 255), -1)

                cls_name = get_class_name(class_names, tracked_cls)
                if tracked_conf is not None:
                    label = f"{cls_name} conf={tracked_conf:.2f}"
                else:
                    label = cls_name

                cv2.putText(
                    viewL,
                    label,
                    (dx1, max(20, dy1 - 8)),
                    cv2.FONT_HERSHEY_SIMPLEX,
                    0.65,
                    (0, 255, 0),
                    2
                )

                if valid_disp.size >= MIN_VALID_DISP_PIXELS:
                    # Trim disparity outliers before taking the median
                    p_low, p_high = np.percentile(valid_disp, [TRIM_PERCENT_LOW,
TRIM_PERCENT_HIGH])
                    trimmed_disp = valid_disp[(valid_disp >= p_low) & (valid_disp <=
p_high)]

                    if trimmed_disp.size > 0:
                        disp_med = float(np.median(trimmed_disp))
                    else:
                        disp_med = float(np.median(valid_disp))

                    # XYZ in mm (processing-frame coordinates)
```

```
                    Z_mm = (fx_p * baseline_mm) / disp_med
                    X_mm = ((cx - cx0_p) * Z_mm) / fx_p
                    Y_mm = ((cy - cy0_p) * Z_mm) / fy_p

                    # Smooth XYZ with short median history
                    x_hist.append(X_mm)
                    y_hist.append(Y_mm)
                    z_hist.append(Z_mm)

                    X_mm_s = median_or_none(x_hist)
                    Y_mm_s = median_or_none(y_hist)
                    Z_mm_s = median_or_none(z_hist)

                    # Convert to inches first
                    X_in = X_mm_s * MM_TO_IN
                    Y_in = Y_mm_s * MM_TO_IN
                    Z_in = Z_mm_s * MM_TO_IN

                    # Round for display/output in 0.5-foot increments
                    X_ft = -round_to_half_foot(X_in)
                    Y_ft = -round_to_half_foot(Y_in)
                    Z_ft = round_to_half_foot(Z_in)

                    info_lines = [
                        f"X = {X_ft:.1f} ft",
                        f"Y = {Y_ft:.1f} ft",
                        f"Z = {Z_ft:.1f} ft",
                        f"FPS = {fps_ema:.1f}"
                    ]

                    if SHOW_DEBUG_DEPTH_TEXT:
                        info_lines.insert(3, f"disp = {disp_med:.2f}")
                        info_lines.insert(4, f"n = {valid_disp.size}")

                    xyz_text_terminal = f"X = {X_ft:.1f} ft | Y = {Y_ft:.1f} ft | Z =
{Z_ft:.1f} ft"
                else:
                    info_lines = [
                        "Drone detected",
                        "Depth unavailable",
                        f"FPS = {fps_ema:.1f}"
                    ]
            else:
                x_hist.clear()
                y_hist.clear()
                z_hist.clear()
        else:
            x_hist.clear()
```

```
            y_hist.clear()
            z_hist.clear()
            info_lines = [
                "No drone detected",
                f"FPS = {fps_ema:.1f}"
            ]

        # Overlay text on left image
        y_text = 30
        for line in info_lines:
            cv2.putText(
                viewL,
                line,
                (20, y_text),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.7,
                (255, 255, 255),
                2
            )
            y_text += 28

        # Resize for display
        left_disp = cv2.resize(viewL, (DISPLAY_W // 2, DISPLAY_H),
interpolation=cv2.INTER_LINEAR)
        right_disp = cv2.resize(viewDisp, (DISPLAY_W // 2, DISPLAY_H),
interpolation=cv2.INTER_LINEAR)

        combined = cv2.hconcat([left_disp, right_disp])
        cv2.putText(
            combined,
            FOOTER_TEXT,
            (20, DISPLAY_H - 20),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.7,
            (255, 255, 255),
            2
        )

        cv2.imshow(WINDOW_NAME, combined)

        if xyz_text_terminal is not None:
            print(xyz_text_terminal, end="\r")

        key = cv2.waitKey(1) & 0xFF
        if key == ord('q'):
            break
        if key == ord('r'):
            print("\nReopening cameras...")
```

```
            left.reopen()
            right.reopen()

    left.stop()
    right.stop()
    cv2.destroyAllWindows()
    print("\nDone.")

if __name__ == "__main__":
    main()
```